

**BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1**



**ĐỒ ÁN KẾT THÚC HỌC PHẦN
HỌC PHẦN: IoT Và Ứng dụng
ĐỀ TÀI: XÂY DỰNG HỆ THỐNG CẢM BIẾN MÔI TRƯỜNG**

Giảng viên hướng dẫn : Ts. Nguyễn Quốc Uy
Nhóm môn học : 11
Họ và tên : Mai Anh Đức
Mã sinh viên : B23DCCN172

HÀ NỘI, 2026

MỤC LỤC

DANH SÁCH HÌNH ẢNH.....	1
DANH SÁCH BẢNG BIỂU.....	2
CHƯƠNG 1. GIỚI THIỆU VỀ HỆ THỐNG.....	3
1.1. Bối cảnh của đề tài.....	3
1.2. Lý do lựa chọn đề tài.....	4
1.3. Bài toán đặt ra.....	7
1.3.1. Bài toán giám sát dữ liệu môi trường.....	7
1.3.2. Bài toán điều khiển thiết bị.....	8
1.4. Mục tiêu của đề tài.....	9
1.5. Kiến trúc tổng quát của hệ thống.....	11
1.6. Các công nghệ được sử dụng.....	14
1.6.1. ESP8266/ESP32.....	14
1.6.2. Các cảm biến môi trường.....	15
1.6.3. MQTT.....	16
1.6.4. Java Spring Boot.....	16
1.6.5. RESTful API.....	17
1.6.6. React.....	17
1.6.7. PostgreSQL.....	18
1.7. Chức năng chính của hệ thống.....	19
1.8. Ý nghĩa thực tiễn của đề tài.....	20
1.9. Ý nghĩa về mặt học tập và nghiên cứu.....	21
1.10. Khả năng mở rộng của hệ thống.....	22
1.11. Công nghệ sử dụng trong đề tài.....	24
1.12. Kết luận chương.....	24
CHƯƠNG 2. THIẾT KẾ TỔNG QUAN.....	26
I. Phân tích lớp thực thể.....	26
1. Phân tích các lớp thực thể.....	26
2. Phân tích mối quan hệ giữa các lớp thực thể.....	32
II. Sơ đồ ERD.....	33
III. THIẾT KẾ TỔNG QUAN CÁC THÀNH PHẦN CỦA HỆ THỐNG.....	33
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	40
I. Xác thực và phân quyền.....	40
1. Đăng nhập.....	40
2. Đăng ký tài khoản.....	47
3. Profile.....	52
II. Giám sát cảm biến.....	60

1.	Xem dữ liệu cảm biến.....	60
2.	Xem biểu đồ.....	69
3.	Xem lịch sử cảm biến.....	75
III.	Điều khiển thiết bị.....	81
1.	Bật tắt thiết bị.....	81
2.	Xem lịch sử điều khiển thiết bị.....	89
CHƯƠNG 4. TỔNG KẾT VÀ LỜI CẢM ƠN.....		95
4.1.	Tổng kết.....	95
4.2.	Lời cảm ơn.....	97

DANH SÁCH HÌNH ẢNH

Hình 1.1	Kiến trúc tổng quan của hệ thống	14
Hình 1.2	Kiến trúc phần cứng	20
Hình 2.1	Sơ đồ ERD	33
Hình 3.1	API Login	41
Hình 3.2	Giao diện Login	45
Hình 3.3	Sequence Diagram Login	46
Hình 3.4	API Đăng ký tài khoản	49
Hình 3.5	Giao diện đăng ký tài khoản	50
Hình 3.6	Sequence Diagram Đăng ký tài khoản	51
Hình 3.7	API Profile	55
Hình 3.8	API Change Password	55
Hình 3.9	API Me	56
Hình 3.10	Giao diện Profile	59
Hình 3.11	Sequence Diagram Profile	59
Hình 3.12	API Xem dữ liệu cảm biến	65
Hình 3.13	Giao diện hiển thị dữ liệu cảm biến	67
Hình 3.14	Sequence Diagram Hiển thị dữ liệu cảm biến	67
Hình 3.15	API Xem Biểu Đồ	70
Hình 3.16	Giao diện xem biểu đồ dữ liệu cảm biến	73
Hình 3.17	Sequence Diagram Xem biểu đồ	74
Hình 3.18	API Lấy dữ liệu cảm biến	76
Hình 3.19	Giao diện xem lịch sử cảm biến	79
Hình 3.20	Sequence Diagram Xem dữ liệu cảm biến	80
Hình 3.21	API điều khiển thiết bị	84
Hình 3.22	Giao diện điều khiển thiết bị	87
Hình 3.23	Sequence Diagram điều khiển thiết bị	88
Hình 3.24	API xem lịch sử điều khiển thiết bị	91
Hình 3.25	Giao diện xem lịch sử điều khiển thiết bị	93
Hình 3.26	Sequence Diagram xem lịch sử điều khiển thiết bị	94

DANH SÁCH BẢNG BIỂU

Bảng 2.1 Các thuộc tính của lớp User	26
Bảng 2.2 Các thuộc tính của lớp Device	27
Bảng 2.3 Thuộc tính của lớp Sensor	28
Bảng 2.4 Thuộc tính của lớp SensorData	29
Bảng 2.5 Thuộc tính của lớp Action	30
Bảng 3.1 Thông tin API đăng nhập	41
Bảng 3.2 Tham số Request của API đăng nhập	41
Bảng 3.3 Các trường Response đăng nhập thành công	42
Bảng 3.4 Các mã phản hồi của API đăng nhập	43
Bảng 3.5 Thông tin API thông tin cá nhân	56
Bảng 3.6 Tham số Request của API Profile	56
Bảng 3.7 Các trường Response API Profile thành công	57
Bảng 3.8 Các mã phản hồi của API Profile	57
Bảng 3.9 Các loại cảnh báo của hệ thống	63
Bảng 3.10 Thông tin API hiển thị dữ liệu của cảm biến	65
Bảng 3.11 Các mã phản hồi của API đo dữ liệu cảm biến	66
Bảng 3.12 Thông tin API xem biểu đồ	70
Bảng 3.13 Tham số Request của API xem biểu đồ	70
Bảng 3.14 Các trường dữ liệu trả về API xem biểu đồ	71
Bảng 3.15 Các mã trả về của API xem biểu đồ	72
Bảng 3.16 Thông tin API xem lịch sử dữ liệu cảm biến	76
Bảng 3.17 Tham số Request API xem lịch sử dữ liệu cảm biến	77
Bảng 3.18 Các mã trả về của API xem dữ liệu cảm biến	78
Bảng 3.19 Thông tin API lấy thông tin trạng thái của thiết bị	84
Bảng 3.20 Các mã trả về API lấy trạng thái thiết bị	84
Bảng 3.21 Thông tin API điều khiển thiết bị	85
Bảng 3.22 Các mã trả về của API điều khiển thiết bị	86
Bảng 3.23 Thông tin API xem lịch sử điều khiển thiết bị	91
Bảng 3.24 Tham số Request của API xem lịch sử điều khiển thiết bị	91
Bảng 3.25 Các mã trả về của API xem lịch sử điều khiển thiết bị	92

CHƯƠNG 1. GIỚI THIỆU VỀ HỆ THỐNG

1.1. Bối cảnh của đề tài

Trong những năm gần đây, cùng với sự phát triển mạnh mẽ của công nghệ thông tin, Internet và các hệ thống nhúng, **Internet of Things (IoT – Internet vạn vật)** đã trở thành một trong những xu hướng công nghệ quan trọng và được ứng dụng

rộng rãi trong nhiều lĩnh vực của đời sống. IoT cho phép các thiết bị vật lý như cảm biến, thiết bị điện tử, máy móc và các thiết bị gia dụng có khả năng kết nối với Internet, trao đổi dữ liệu và tương tác với các hệ thống phần mềm một cách tự động.

Nếu như trước đây các thiết bị điện tử thường hoạt động độc lập và yêu cầu người dùng phải trực tiếp thao tác tại vị trí đặt thiết bị thì với sự phát triển của IoT, người dùng có thể theo dõi, giám sát và điều khiển thiết bị từ xa thông qua Internet. Các thiết bị IoT có khả năng liên tục thu thập dữ liệu từ môi trường, truyền dữ liệu đến máy chủ để xử lý, lưu trữ và cung cấp thông tin cho người dùng thông qua các ứng dụng Web hoặc ứng dụng di động.

Nhờ khả năng kết nối và trao đổi dữ liệu gần như theo thời gian thực, IoT hiện nay được ứng dụng trong nhiều lĩnh vực như nhà thông minh, thành phố thông minh, nông nghiệp thông minh, giám sát môi trường, y tế, giao thông, sản xuất công nghiệp, quản lý năng lượng và tự động hóa.

Một trong những bài toán phổ biến của IoT là **giám sát các thông số môi trường và điều khiển thiết bị từ xa**. Trong thực tế, các thông số như nhiệt độ, độ ẩm và cường độ ánh sáng có ảnh hưởng trực tiếp đến môi trường sống, môi trường làm việc cũng như nhiều hoạt động sản xuất.

Ví dụ, trong một căn phòng, việc theo dõi nhiệt độ và độ ẩm giúp người sử dụng đánh giá được điều kiện môi trường hiện tại để đưa ra quyết định bật hoặc tắt các thiết bị như quạt, điều hòa hoặc máy tạo độ ẩm. Trong lĩnh vực nông nghiệp, nhiệt độ, độ ẩm và ánh sáng là những yếu tố ảnh hưởng trực tiếp đến quá trình sinh trưởng và phát triển của cây trồng. Trong các nhà kho hoặc khu vực bảo quản hàng hóa, việc theo dõi liên tục các thông số môi trường giúp phát hiện sớm những điều kiện bất thường có thể ảnh hưởng đến chất lượng sản phẩm.

Tuy nhiên, nếu việc thu thập các thông số môi trường được thực hiện hoàn toàn thủ công thì sẽ tồn tại nhiều hạn chế. Người dùng phải trực tiếp đến vị trí đặt thiết bị để kiểm tra, dữ liệu không được cập nhật liên tục và khó theo dõi sự thay đổi trong

một khoảng thời gian dài. Ngoài ra, khi người dùng không có mặt tại vị trí đặt thiết bị, việc kiểm tra trạng thái hoặc điều khiển thiết bị cũng trở nên khó khăn.

Chính vì vậy, việc xây dựng một hệ thống có khả năng **tự động thu thập dữ liệu môi trường, truyền dữ liệu qua mạng, lưu trữ dữ liệu trên máy chủ và cung cấp giao diện giám sát, điều khiển thiết bị từ xa** là một bài toán có ý nghĩa thực tiễn cao.

Xuất phát từ nhu cầu đó, nhóm lựa chọn xây dựng đề tài **“Hệ thống IoT giám sát môi trường và điều khiển thiết bị từ xa”**.

1.2. Lý do lựa chọn đề tài

Việc lựa chọn đề tài xuất phát từ cả nhu cầu thực tế và mục tiêu tìm hiểu, vận dụng các kiến thức liên quan đến Internet of Things, lập trình thiết bị nhúng, giao tiếp mạng, phát triển Backend, xây dựng giao diện Web và quản lý cơ sở dữ liệu.

Trước hết, hệ thống giám sát môi trường là một mô hình phù hợp để nghiên cứu kiến trúc cơ bản của một hệ thống IoT hoàn chỉnh. Một hệ thống IoT không chỉ bao gồm các cảm biến hoặc vi điều khiển mà còn là sự kết hợp giữa nhiều thành phần như thiết bị phần cứng, giao thức truyền thông, MQTT Broker, Backend, cơ sở dữ liệu và giao diện người dùng.

Thông qua đề tài, nhóm có thể xây dựng và kiểm thử toàn bộ quá trình truyền dữ liệu từ thiết bị vật lý đến người sử dụng theo mô hình:

Cảm biến → ESP8266/ESP32 → MQTT Broker → Spring Boot Backend → PostgreSQL → React Frontend → Người dùng.

Đồng thời, hệ thống hỗ trợ chiều giao tiếp ngược lại để người dùng có thể điều khiển thiết bị:

Người dùng → React Frontend → Spring Boot Backend → MQTT Broker → ESP8266/ESP32 → Thiết bị.

Như vậy, thay vì chỉ xây dựng một hệ thống thu thập dữ liệu theo một chiều, đề tài hướng tới xây dựng hệ thống IoT có khả năng **giao tiếp hai chiều**. Thiết bị vừa có thể gửi dữ liệu cảm biến lên hệ thống, vừa có khả năng nhận lệnh điều khiển từ Backend và phản hồi trạng thái thực tế sau khi thực hiện lệnh.

Một lý do khác để nhóm lựa chọn đề tài là các thành phần phần cứng và phần mềm được sử dụng đều phổ biến, có tính thực tiễn cao và có khả năng mở rộng cho nhiều bài toán khác nhau.

ESP8266 và ESP32 là các vi điều khiển được sử dụng rộng rãi trong các dự án IoT nhờ khả năng tích hợp Wi-Fi, kích thước nhỏ, chi phí thấp nhưng vẫn có khả năng xử lý phù hợp với nhiều ứng dụng. Trong hệ thống, ESP8266/ESP32 được kết nối với các cảm biến để thu thập dữ liệu thực tế từ môi trường.

Các thông số chính được hệ thống theo dõi bao gồm:

- Nhiệt độ môi trường.
- Độ ẩm không khí.
- Cường độ ánh sáng.

Sau khi đọc dữ liệu từ cảm biến, ESP8266/ESP32 tiến hành xử lý và đóng gói dữ liệu theo định dạng phù hợp trước khi gửi đến hệ thống thông qua giao thức MQTT.

MQTT được lựa chọn làm giao thức truyền thông giữa thiết bị IoT và Backend do đây là một giao thức truyền tin nhẹ, phù hợp với các thiết bị có tài nguyên phần cứng hạn chế và những hệ thống cần trao đổi dữ liệu thường xuyên qua mạng.

MQTT hoạt động theo mô hình **Publish/Subscribe**. ESP8266/ESP32 có thể đóng vai trò Publisher để gửi dữ liệu cảm biến lên một Topic. Spring Boot Backend đăng ký Subscribe Topic tương ứng để nhận dữ liệu. Ngược lại, khi người dùng cần điều khiển thiết bị, Backend đóng vai trò Publisher gửi lệnh đến Topic điều khiển và ESP8266/ESP32 Subscribe Topic đó để tiếp nhận lệnh.

Thông qua cơ chế trên, thiết bị và Backend không cần kết nối trực tiếp chặt chẽ với nhau mà có thể trao đổi dữ liệu thông qua MQTT Broker. Điều này giúp hệ thống linh hoạt hơn và tạo điều kiện thuận lợi khi cần mở rộng số lượng thiết bị trong tương lai.

Bên cạnh phần cứng và MQTT, đề tài còn giúp nhóm xây dựng một hệ thống phần mềm hoàn chỉnh từ Backend đến Frontend.

Backend được xây dựng bằng **Java Spring Boot**. Spring Boot là một framework phổ biến trong hệ sinh thái Java, cung cấp nhiều công cụ hỗ trợ xây dựng các ứng dụng Web và RESTful API. Trong hệ thống, Spring Boot đảm nhiệm vai trò xử lý nghiệp vụ, tiếp nhận dữ liệu từ MQTT, lưu trữ dữ liệu vào PostgreSQL, cung cấp RESTful API cho Frontend và gửi lệnh điều khiển tới thiết bị.

Phía giao diện người dùng được xây dựng bằng **React**. React cho phép xây dựng giao diện Web hiện đại theo hướng component, giúp các thành phần giao diện dễ quản lý, tái sử dụng và cập nhật khi dữ liệu thay đổi.

Thông qua giao diện Web, người dùng có thể theo dõi các thông số môi trường hiện tại, quan sát biểu đồ biến động của dữ liệu, tra cứu lịch sử dữ liệu cảm biến, theo dõi trạng thái thiết bị và thực hiện các thao tác bật hoặc tắt thiết bị.

Để lưu trữ dữ liệu lâu dài, hệ thống sử dụng **PostgreSQL** làm hệ quản trị cơ sở dữ liệu. Cơ sở dữ liệu được sử dụng để lưu dữ liệu cảm biến, thông tin thiết bị, trạng thái thiết bị và lịch sử điều khiển.

Như vậy, đề tài là sự kết hợp của nhiều thành phần khác nhau từ phần cứng, mạng truyền thông, Backend, Frontend cho đến cơ sở dữ liệu. Việc xây dựng hệ thống giúp nhóm hiểu rõ hơn cách các thành phần phối hợp với nhau trong một hệ thống IoT hoàn chỉnh.

1.3. Bài toán đặt ra

Bài toán chính của đề tài là xây dựng một hệ thống IoT có khả năng theo dõi các thông số môi trường và điều khiển thiết bị từ xa thông qua Internet.

Hệ thống cần giải quyết hai nhóm bài toán chính là:

- Giám sát dữ liệu môi trường.
- Điều khiển thiết bị từ xa.

1.3.1. Bài toán giám sát dữ liệu môi trường

Các cảm biến được kết nối với ESP8266/ESP32 có nhiệm vụ đo các thông số môi trường theo một chu kỳ nhất định.

Sau khi thu thập dữ liệu, ESP8266/ESP32 tiến hành xử lý và tạo dữ liệu bao gồm các thông tin như:

- Giá trị nhiệt độ.
- Giá trị độ ẩm.
- Giá trị ánh sáng.
- Thời gian ghi nhận dữ liệu.

Dữ liệu sau đó được ESP8266/ESP32 Publish đến MQTT Broker thông qua Topic dữ liệu cảm biến.

Spring Boot Backend đăng ký Subscribe Topic tương ứng trên MQTT Broker. Khi có dữ liệu mới, Backend tiếp nhận bản tin, kiểm tra định dạng, xử lý dữ liệu và lưu thông tin vào PostgreSQL.

Các dữ liệu đã lưu trong cơ sở dữ liệu có thể được Frontend truy xuất thông qua các RESTful API do Spring Boot cung cấp.

React Frontend sử dụng các API để lấy dữ liệu và hiển thị cho người dùng dưới nhiều hình thức như:

- Giá trị cảm biến hiện tại.
- Danh sách dữ liệu cảm biến.

- Biểu đồ nhiệt độ.
- Biểu đồ độ ẩm.
- Biểu đồ cường độ ánh sáng.
- Lịch sử dữ liệu theo thời gian.

Nhờ đó, người dùng không cần trực tiếp quan sát các cảm biến hoặc Serial Monitor mà vẫn có thể theo dõi tình trạng môi trường thông qua giao diện Web.

1.3.2. Bài toán điều khiển thiết bị

Ngoài chức năng giám sát dữ liệu, hệ thống còn cần cho phép người dùng gửi lệnh điều khiển từ xa đến các thiết bị IoT.

Trong phạm vi đề tài, các đèn LED được sử dụng làm thiết bị đầu ra, đại diện cho các thiết bị điện có thể được bật hoặc tắt.

Khi người dùng thực hiện thao tác bật hoặc tắt thiết bị trên giao diện React, Frontend gửi yêu cầu đến RESTful API của Spring Boot Backend.

Backend tiếp nhận yêu cầu, kiểm tra thiết bị và trạng thái hiện tại, sau đó tạo lệnh điều khiển tương ứng và Publish lệnh lên MQTT Broker.

ESP8266/ESP32 đã Subscribe Topic điều khiển nên sẽ nhận được bản tin do Backend gửi tới. Sau khi nhận lệnh, vi điều khiển tiến hành điều khiển chân GPIO tương ứng để bật hoặc tắt LED.

Ví dụ:

Bật LED:

React → Spring Boot → MQTT Broker → ESP32 → GPIO HIGH → LED sáng.

Tắt LED:

React → Spring Boot → MQTT Broker → ESP32 → GPIO LOW → LED tắt.

Sau khi thực hiện thao tác, ESP8266/ESP32 gửi trạng thái thực tế của thiết bị trở lại MQTT Broker. Spring Boot tiếp nhận bản tin phản hồi và cập nhật trạng thái thiết bị trong hệ thống.

Quá trình giao tiếp hai chiều có thể được mô tả như sau:

Frontend → Backend → MQTT Broker → ESP8266/ESP32 → Thiết bị

và:

Thiết bị → ESP8266/ESP32 → MQTT Broker → Backend → Frontend.

Các thao tác điều khiển được lưu vào PostgreSQL nhằm phục vụ việc theo dõi và tra cứu lịch sử hoạt động của thiết bị.

1.4. Mục tiêu của đề tài

Mục tiêu tổng quát của đề tài là nghiên cứu và xây dựng một hệ thống IoT có khả năng **thu thập dữ liệu môi trường, truyền dữ liệu thông qua MQTT, xử lý và lưu trữ dữ liệu tại Backend, hiển thị dữ liệu trên giao diện Web và cho phép người dùng điều khiển thiết bị từ xa.**

Từ mục tiêu tổng quát trên, đề tài đặt ra các mục tiêu cụ thể như sau:

Thứ nhất, xây dựng thiết bị IoT dựa trên ESP8266/ESP32 có khả năng kết nối với các cảm biến và thu thập các thông số môi trường.

Thiết bị cần đọc dữ liệu định kỳ từ cảm biến nhiệt độ, độ ẩm và ánh sáng, sau đó đóng gói dữ liệu thành định dạng phù hợp để gửi qua mạng.

Thứ hai, thiết lập kết nối giữa ESP8266/ESP32 và MQTT Broker. Vì điều khiển cần có khả năng:

- Kết nối Wi-Fi.
- Kết nối MQTT Broker.
- Publish dữ liệu cảm biến.

- Subscribe Topic điều khiển.
- Tiếp nhận và thực hiện lệnh điều khiển.
- Gửi phản hồi trạng thái thiết bị.

Thứ ba, xây dựng Backend bằng **Spring Boot** có khả năng giao tiếp với MQTT Broker, PostgreSQL và React Frontend.

Backend đảm nhiệm các chức năng chính như:

- Nhận dữ liệu cảm biến từ MQTT.
- Kiểm tra và xử lý dữ liệu.
- Lưu dữ liệu cảm biến vào PostgreSQL.
- Truy vấn dữ liệu từ cơ sở dữ liệu.
- Xây dựng RESTful API.
- Tiếp nhận yêu cầu điều khiển từ Frontend.
- Publish lệnh điều khiển thông qua MQTT.
- Nhận phản hồi trạng thái từ thiết bị.
- Lưu lịch sử thao tác điều khiển.

Thứ tư, xây dựng cơ sở dữ liệu PostgreSQL để quản lý dữ liệu của hệ thống một cách có tổ chức.

Các đối tượng chính cần quản lý bao gồm:

- Thiết bị.
- Dữ liệu cảm biến.
- Trạng thái thiết bị.
- Lịch sử điều khiển.

Thứ năm, xây dựng giao diện Web bằng **React** nhằm cung cấp phương thức tương tác trực quan cho người dùng.

Giao diện cần hỗ trợ:

- Theo dõi nhiệt độ.

- Theo dõi độ ẩm.
- Theo dõi cường độ ánh sáng.
- Xem biểu đồ biến động dữ liệu.
- Xem lịch sử dữ liệu cảm biến.
- Theo dõi trạng thái thiết bị.
- Bật/tắt thiết bị từ xa.
- Xem lịch sử điều khiển.

Cuối cùng, hệ thống cần có cấu trúc rõ ràng và khả năng mở rộng để có thể bổ sung thêm nhiều cảm biến, thiết bị hoặc chức năng trong tương lai.

1.5. Kiến trúc tổng quát của hệ thống

Hệ thống được thiết kế theo kiến trúc gồm nhiều thành phần. Mỗi thành phần đảm nhiệm một nhiệm vụ riêng nhưng phối hợp với nhau để tạo thành một hệ thống IoT hoàn chỉnh.

Kiến trúc tổng thể có thể mô tả như sau:

Cảm biến → ESP8266/ESP32 → MQTT Broker → Spring Boot → PostgreSQL

và:

Spring Boot ↔ RESTful API ↔ React Web

Đối với quá trình thu thập dữ liệu cảm biến, luồng xử lý chính gồm:

Bước 1: Cảm biến thu thập các thông số từ môi trường.

Bước 2: ESP8266/ESP32 đọc giá trị từ cảm biến.

Bước 3: Vi điều khiển xử lý và đóng gói dữ liệu.

Bước 4: ESP8266/ESP32 Publish dữ liệu lên MQTT Broker.

Bước 5: Spring Boot Subscribe Topic dữ liệu và tiếp nhận bản tin.

Bước 6: Backend kiểm tra và xử lý dữ liệu.

Bước 7: Backend lưu dữ liệu vào PostgreSQL.

Bước 8: React gửi yêu cầu đến RESTful API.

Bước 9: Spring Boot truy vấn dữ liệu từ PostgreSQL.

Bước 10: Backend trả dữ liệu về Frontend.

Bước 11: React hiển thị dữ liệu dưới dạng số liệu, danh sách hoặc biểu đồ.

Đối với quá trình điều khiển thiết bị:

Bước 1: Người dùng thực hiện thao tác bật/tắt thiết bị trên giao diện React.

Bước 2: React gửi HTTP Request đến Spring Boot.

Bước 3: Backend kiểm tra thông tin thiết bị.

Bước 4: Backend tạo bản tin MQTT điều khiển.

Bước 5: Spring Boot Publish bản tin lên MQTT Broker.

Bước 6: ESP8266/ESP32 nhận bản tin từ Topic điều khiển.

Bước 7: ESP8266/ESP32 thực hiện thay đổi trạng thái GPIO.

Bước 8: Thiết bị được bật hoặc tắt.

Bước 9: ESP8266/ESP32 gửi trạng thái thực tế trở lại MQTT Broker.

Bước 10: Spring Boot tiếp nhận phản hồi.

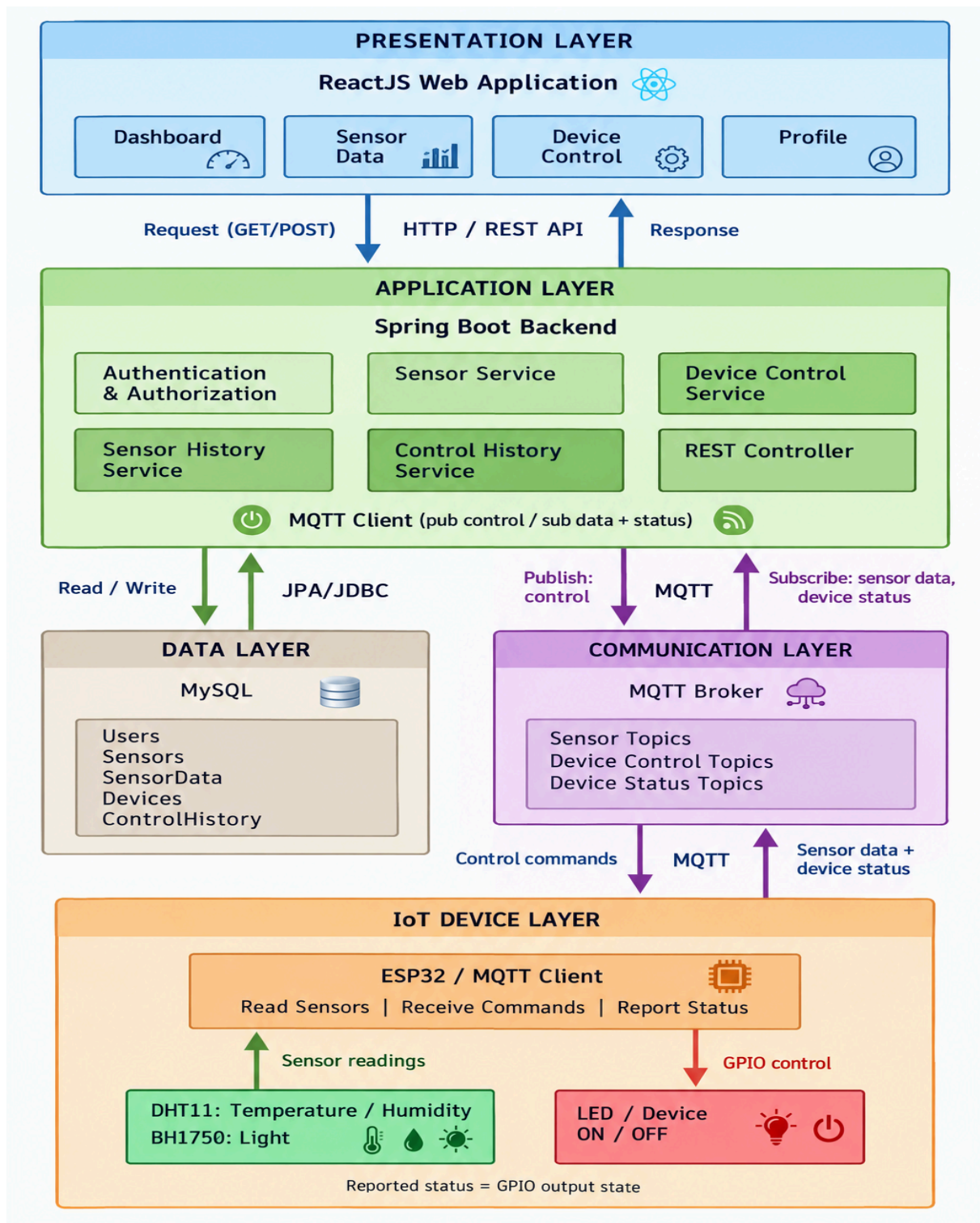
Bước 11: Backend cập nhật trạng thái và lưu lịch sử điều khiển.

Bước 12: React cập nhật trạng thái thiết bị trên giao diện.

Việc thiết kế theo mô hình trên giúp phân chia rõ trách nhiệm giữa các thành phần.

Thiết bị IoT chịu trách nhiệm thu thập dữ liệu và thực thi lệnh, Backend thực hiện xử

lý nghiệp vụ và quản lý dữ liệu, trong khi Frontend tập trung vào việc hiển thị thông tin và tương tác với người dùng.



Hình 1.1 Kiến trúc tổng quan của hệ thống

1.6. Các công nghệ được sử dụng

1.6.1. ESP8266/ESP32

ESP8266 và ESP32 là các dòng vi điều khiển được sử dụng phổ biến trong lĩnh vực IoT. Ưu điểm quan trọng của các vi điều khiển này là tích hợp khả năng kết nối Wi-Fi, giúp thiết bị có thể kết nối trực tiếp với mạng không dây và giao tiếp với MQTT Broker.

Trong hệ thống, ESP8266/ESP32 đảm nhiệm các nhiệm vụ:

- Kết nối Wi-Fi.
- Kết nối MQTT Broker.
- Đọc dữ liệu cảm biến.
- Xử lý dữ liệu cảm biến.
- Publish dữ liệu lên MQTT.
- Subscribe Topic điều khiển.
- Nhận lệnh bật/tắt thiết bị.
- Điều khiển các chân GPIO.
- Gửi phản hồi trạng thái thiết bị.

Việc sử dụng ESP8266/ESP32 giúp hệ thống có chi phí phần cứng thấp nhưng vẫn đáp ứng tốt các yêu cầu của một hệ thống IoT cơ bản.

1.6.2. Các cảm biến môi trường

Các cảm biến là thành phần trực tiếp thu thập thông tin từ môi trường thực tế.

Trong phạm vi đề tài, hệ thống thu thập ba thông số chính:

- Nhiệt độ.
- Độ ẩm.
- Cường độ ánh sáng.

Dữ liệu từ các cảm biến được ESP8266/ESP32 đọc theo chu kỳ. Sau mỗi lần đọc, các giá trị được đóng gói thành một bản tin dữ liệu và gửi lên MQTT Broker.

Việc kết hợp nhiều loại cảm biến giúp hệ thống cung cấp cái nhìn tổng thể hơn về điều kiện môi trường thay vì chỉ giám sát một thông số duy nhất.

1.6.3. MQTT

MQTT là giao thức truyền thông theo mô hình **Publish/Subscribe**, được sử dụng rộng rãi trong các hệ thống IoT.

Ba thành phần quan trọng của MQTT gồm:

Publisher: Thành phần gửi bản tin.

Subscriber: Thành phần đăng ký nhận bản tin.

Broker: Thành phần trung gian nhận và phân phối bản tin.

Trong hệ thống:

Khi gửi dữ liệu cảm biến:

ESP8266/ESP32 → Publish → MQTT Broker → Spring Boot Subscribe.

Khi gửi lệnh điều khiển:

Spring Boot → Publish → MQTT Broker → ESP8266/ESP32 Subscribe.

Việc sử dụng MQTT giúp giảm sự phụ thuộc giữa thiết bị và Backend, đồng thời hỗ trợ tốt cho việc trao đổi dữ liệu thường xuyên.

1.6.4. Java Spring Boot

Spring Boot là framework được sử dụng để phát triển Backend của hệ thống.

Spring Boot giúp đơn giản hóa quá trình xây dựng ứng dụng Java Web, hỗ trợ cấu hình tự động và tích hợp thuận tiện với nhiều công nghệ khác nhau.

Trong hệ thống, Spring Boot có nhiệm vụ:

- Kết nối với MQTT Broker.

- Subscribe dữ liệu cảm biến.
- Publish lệnh điều khiển.
- Xử lý logic nghiệp vụ.
- Kết nối PostgreSQL.
- Lưu và truy vấn dữ liệu.
- Cung cấp RESTful API cho React.
- Quản lý trạng thái thiết bị.
- Lưu lịch sử điều khiển.

Việc lựa chọn Spring Boot giúp Backend có cấu trúc rõ ràng và thuận tiện cho việc bảo trì, mở rộng hệ thống trong tương lai.

1.6.5. RESTful API

RESTful API được sử dụng làm phương thức giao tiếp giữa React Frontend và Spring Boot Backend.

Thông qua API, Frontend có thể gửi yêu cầu tới Backend để truy xuất dữ liệu hoặc thực hiện các thao tác điều khiển.

Hệ thống có thể cung cấp các nhóm API như:

- API lấy dữ liệu cảm biến.
- API lấy dữ liệu cảm biến mới nhất.
- API lấy lịch sử cảm biến.
- API lấy danh sách thiết bị.
- API lấy trạng thái thiết bị.
- API bật/tắt thiết bị.
- API lấy lịch sử điều khiển.

Việc sử dụng RESTful API giúp Frontend và Backend được tách biệt, thuận tiện cho việc phát triển và kiểm thử từng thành phần.

1.6.6. React

React là thư viện JavaScript được sử dụng để xây dựng giao diện Web của hệ thống.

React hỗ trợ xây dựng giao diện theo hướng component, giúp các thành phần có khả năng tái sử dụng và dễ quản lý.

Giao diện React của hệ thống có thể bao gồm:

- Dashboard tổng quan.
- Thẻ hiển thị nhiệt độ.
- Thẻ hiển thị độ ẩm.
- Thẻ hiển thị ánh sáng.
- Biểu đồ dữ liệu theo thời gian.
- Danh sách lịch sử dữ liệu cảm biến.
- Giao diện điều khiển thiết bị.
- Trạng thái ON/OFF của thiết bị.
- Lịch sử điều khiển.

Nhờ giao diện Web, người dùng có thể theo dõi toàn bộ hệ thống một cách trực quan mà không cần thao tác trực tiếp trên thiết bị vật lý.

1.6.7. PostgreSQL

PostgreSQL được sử dụng làm hệ quản trị cơ sở dữ liệu của hệ thống.

Do dữ liệu cảm biến liên tục phát sinh theo thời gian, việc lưu trữ dữ liệu là cần thiết để phục vụ cho chức năng tra cứu lịch sử và hiển thị biểu đồ.

PostgreSQL được sử dụng để:

- Lưu dữ liệu nhiệt độ.
- Lưu dữ liệu độ ẩm.
- Lưu dữ liệu ánh sáng.
- Lưu thông tin thiết bị.
- Lưu trạng thái thiết bị.
- Lưu lịch sử điều khiển.

- Hỗ trợ truy vấn dữ liệu theo thời gian.

Nhờ đó, hệ thống không chỉ cung cấp thông tin tại thời điểm hiện tại mà còn cho phép người dùng theo dõi sự thay đổi của môi trường trong quá khứ.

1.7. Chức năng chính của hệ thống

Hệ thống được xây dựng với các chức năng chính sau:

Thu thập dữ liệu cảm biến: ESP8266/ESP32 thực hiện đọc dữ liệu từ các cảm biến theo chu kỳ, bao gồm nhiệt độ, độ ẩm và cường độ ánh sáng.

Truyền dữ liệu thông qua MQTT: Sau khi thu thập, dữ liệu được đóng gói thành bản tin và Publish lên MQTT Broker thông qua Topic tương ứng.

Tiếp nhận và xử lý dữ liệu: Spring Boot Subscribe Topic dữ liệu cảm biến để tiếp nhận các bản tin do ESP8266/ESP32 gửi lên. Dữ liệu được kiểm tra và xử lý trước khi lưu vào cơ sở dữ liệu.

Lưu trữ dữ liệu: Dữ liệu cảm biến được lưu vào PostgreSQL để phục vụ cho việc tra cứu và thống kê.

Hiển thị dữ liệu hiện tại: React hiển thị các giá trị nhiệt độ, độ ẩm và ánh sáng hiện tại trên Dashboard.

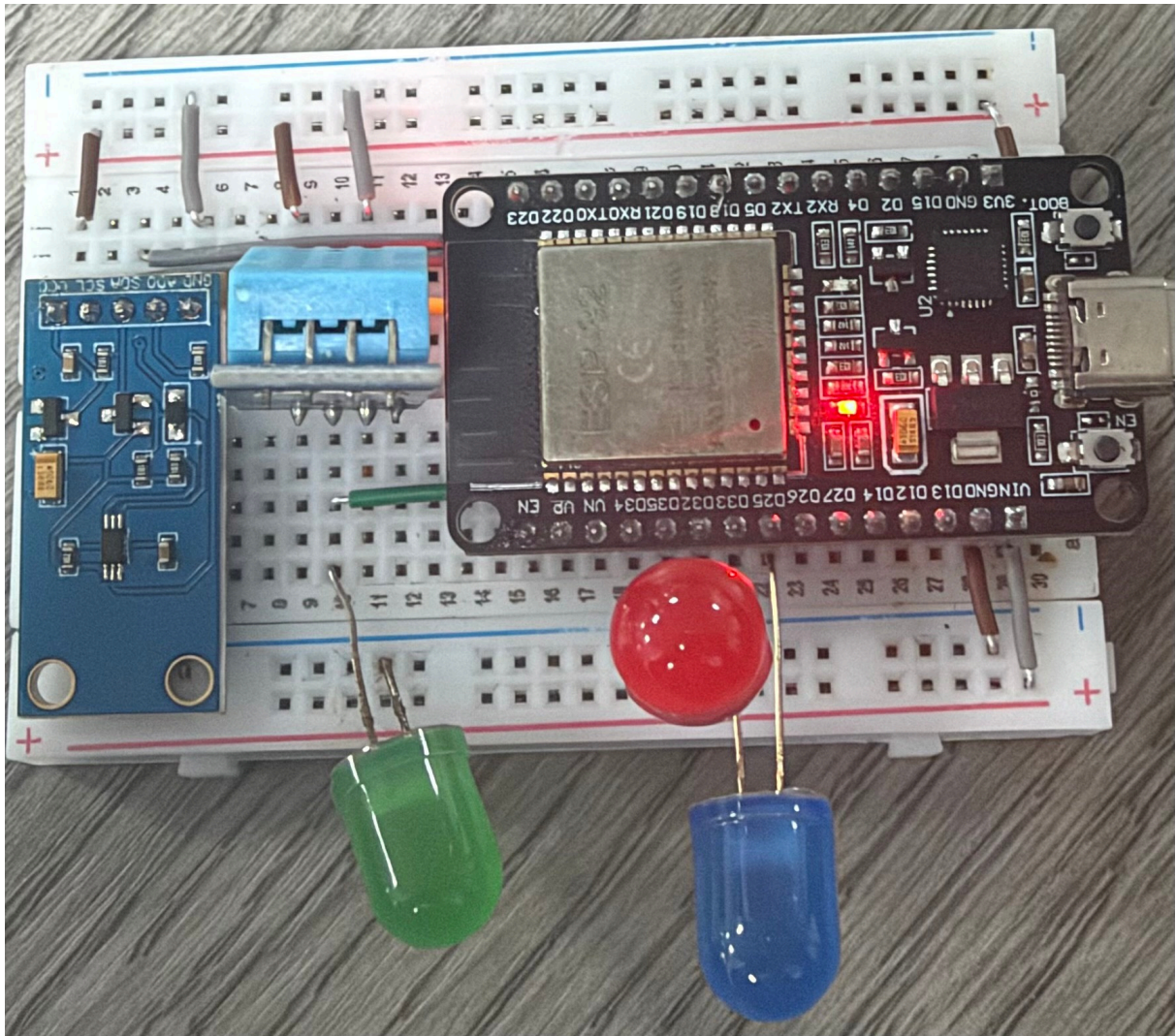
Hiển thị biểu đồ: Dữ liệu cảm biến được biểu diễn dưới dạng biểu đồ theo thời gian nhằm giúp người dùng dễ dàng theo dõi xu hướng thay đổi.

Tra cứu lịch sử: Người dùng có thể xem lại các dữ liệu cảm biến đã được ghi nhận trước đó.

Điều khiển thiết bị: Người dùng có thể thực hiện bật hoặc tắt thiết bị thông qua giao diện Web.

Phản hồi trạng thái thiết bị: Sau khi thiết bị thực hiện lệnh, ESP8266/ESP32 gửi trạng thái thực tế về Backend để đảm bảo giao diện hiển thị đúng trạng thái của thiết bị.

Lưu lịch sử điều khiển: Mỗi thao tác bật hoặc tắt thiết bị được lưu lại trong cơ sở dữ liệu để phục vụ việc theo dõi lịch sử hoạt động.



Hình 1.2 Kiến trúc phần cứng

1.8. Ý nghĩa thực tiễn của đề tài

Đề tài có ý nghĩa thực tiễn vì mô hình được xây dựng có thể được áp dụng và mở rộng cho nhiều bài toán IoT khác nhau.

Đối với **nhà thông minh**, hệ thống có thể được sử dụng để theo dõi nhiệt độ, độ ẩm và ánh sáng trong phòng, đồng thời điều khiển các thiết bị như đèn, quạt hoặc điều hòa.

Đối với **nông nghiệp thông minh**, hệ thống có thể được bổ sung cảm biến độ ẩm đất và các thiết bị như máy bơm nước. Khi đó, người dùng có thể theo dõi điều kiện môi trường và điều khiển hệ thống tưới từ xa.

Đối với **nhà kho**, các cảm biến có thể được sử dụng để giám sát nhiệt độ và độ ẩm nhằm duy trì điều kiện bảo quản phù hợp cho hàng hóa.

Đối với **phòng học hoặc văn phòng**, hệ thống có thể hỗ trợ giám sát môi trường và điều khiển các thiết bị điện.

Trong tương lai, hệ thống cũng có thể phát triển theo hướng tự động hóa. Ví dụ, nếu nhiệt độ vượt quá một ngưỡng nhất định, hệ thống có thể tự động bật quạt. Khi cường độ ánh sáng thấp hơn mức cho phép, hệ thống có thể tự động bật đèn.

Như vậy, hệ thống không chỉ đáp ứng yêu cầu của đề tài mà còn có khả năng phát triển thành một nền tảng cho những ứng dụng IoT phức tạp hơn.

1.9. Ý nghĩa về mặt học tập và nghiên cứu

Bên cạnh ý nghĩa thực tiễn, đề tài còn giúp nhóm vận dụng tổng hợp nhiều kiến thức đã học vào một hệ thống hoàn chỉnh.

Thông qua quá trình thực hiện đề tài, nhóm có cơ hội tìm hiểu và áp dụng các kiến thức về:

- Lập trình vi điều khiển ESP8266/ESP32.
- Kết nối và đọc dữ liệu cảm biến.
- Điều khiển GPIO.
- Kết nối Wi-Fi.
- Giao thức MQTT.
- Mô hình Publish/Subscribe.
- Lập trình mạng.
- Java và Spring Boot.
- Xây dựng RESTful API.
- React.
- Thiết kế cơ sở dữ liệu.
- PostgreSQL.
- Giao tiếp Frontend – Backend.

- Giao tiếp Backend – MQTT.
- Giao tiếp MQTT – ESP8266/ESP32.
- Quản lý trạng thái thiết bị.
- Lưu trữ và xử lý dữ liệu theo thời gian.

Điểm quan trọng của đề tài là giúp nhóm hiểu được cách các thành phần thuộc nhiều lĩnh vực khác nhau kết hợp với nhau để xây dựng một hệ thống hoàn chỉnh.

Thay vì chỉ xây dựng một ứng dụng Web hoặc chỉ lập trình một thiết bị phần cứng, nhóm cần giải quyết đồng thời các vấn đề về phần cứng, mạng truyền thông, giao thức MQTT, Backend, Frontend và cơ sở dữ liệu.

Điều này giúp nâng cao khả năng tư duy hệ thống và hiểu rõ hơn quá trình dữ liệu được truyền từ môi trường vật lý đến giao diện người dùng và ngược lại.

1.10. Khả năng mở rộng của hệ thống

Một trong những ưu điểm của kiến trúc được sử dụng trong đề tài là khả năng mở rộng trong tương lai.

Hiện tại, hệ thống chủ yếu tập trung vào ba thông số môi trường là nhiệt độ, độ ẩm và ánh sáng. Tuy nhiên, hệ thống có thể bổ sung thêm các loại cảm biến như:

- Cảm biến chất lượng không khí.
- Cảm biến khí gas.
- Cảm biến khói.
- Cảm biến độ ẩm đất.
- Cảm biến chuyển động.
- Cảm biến mưa.
- Cảm biến bụi mịn.
- Cảm biến CO₂.

Các thiết bị đầu ra cũng có thể được mở rộng từ các đèn LED sang:

- Quạt điện.

- Đèn chiếu sáng.
- Máy bơm nước.
- Máy điều hòa.
- Máy tạo độ ẩm.
- Còi cảnh báo.
- Relay điều khiển thiết bị điện.

Hệ thống cũng có thể được mở rộng để hỗ trợ nhiều ESP8266/ESP32 hoạt động đồng thời. Mỗi thiết bị có thể được định danh bằng một mã riêng và MQTT Topic được tổ chức theo từng thiết bị, chẳng hạn:

sensor/{deviceId}/data

device/{deviceId}/control

device/{deviceId}/status

Cách tổ chức này giúp Backend xác định dữ liệu thuộc thiết bị nào và gửi lệnh điều khiển chính xác đến thiết bị cần thực hiện.

Ngoài ra, hệ thống có thể được phát triển thêm các chức năng nâng cao như:

- Đăng nhập và xác thực người dùng.
- Phân quyền người dùng.
- Cảnh báo khi dữ liệu vượt ngưỡng.
- Gửi thông báo.
- Điều khiển thiết bị tự động.
- Thiết lập ngưỡng hoạt động.
- Lập lịch bật/tắt thiết bị.
- Thống kê dữ liệu theo ngày, tuần hoặc tháng.
- Quản lý nhiều thiết bị.
- Quản lý nhiều khu vực.
- Xuất báo cáo dữ liệu.
- Tích hợp ứng dụng di động.

- Áp dụng Machine Learning hoặc AI để phân tích và dự đoán dữ liệu cảm biến.

Do đó, hệ thống được xây dựng trong phạm vi đề tài có thể đóng vai trò là nền tảng để tiếp tục phát triển các hệ thống IoT lớn và phức tạp hơn trong tương lai.

1.11. Công nghệ sử dụng trong đề tài

Các công nghệ chính được sử dụng trong quá trình xây dựng hệ thống bao gồm:

- **ESP8266/ESP32:** Là thành phần điều khiển trung tâm phía thiết bị IoT. ESP8266/ESP32 thực hiện kết nối Wi-Fi, đọc dữ liệu cảm biến, gửi dữ liệu lên MQTT Broker, nhận lệnh điều khiển và điều khiển các thiết bị đầu ra.
- **Cảm biến môi trường:** Được sử dụng để đo các thông số vật lý trong môi trường, bao gồm nhiệt độ, độ ẩm và cường độ ánh sáng.
- **MQTT:** Là giao thức truyền thông chính giữa thiết bị IoT và Backend. MQTT hoạt động theo mô hình Publish/Subscribe và phù hợp với các hệ thống IoT cần truyền dữ liệu thường xuyên.
- **MQTT Broker:** Đóng vai trò trung gian tiếp nhận và phân phối bản tin giữa ESP8266/ESP32 và Spring Boot Backend.
- **Java:** Là ngôn ngữ lập trình chính được sử dụng để phát triển Backend.
- **Spring Boot:** Được sử dụng để xây dựng Backend, RESTful API, xử lý nghiệp vụ, kết nối cơ sở dữ liệu và tích hợp với MQTT.
- **React:** Được sử dụng để xây dựng giao diện Web, hiển thị dữ liệu cảm biến, biểu đồ, trạng thái thiết bị và cung cấp chức năng điều khiển.
- **PostgreSQL:** Được sử dụng làm hệ quản trị cơ sở dữ liệu, chịu trách nhiệm lưu trữ dữ liệu cảm biến, thông tin thiết bị và lịch sử điều khiển.

1.12. Kết luận chương

Từ nhu cầu thực tế trong việc theo dõi các thông số môi trường và điều khiển thiết bị từ xa, nhóm lựa chọn xây dựng đề tài “**Hệ thống IoT giám sát môi trường và điều khiển thiết bị từ xa**”.

Đề tài xây dựng một hệ thống IoT với sự kết hợp giữa thiết bị phần cứng và hệ thống phần mềm. ESP8266/ESP32 kết nối với các cảm biến để thu thập nhiệt độ, độ ẩm và cường độ ánh sáng. Dữ liệu được truyền thông qua giao thức MQTT đến Backend được xây dựng bằng Java Spring Boot.

Spring Boot tiếp nhận, xử lý và lưu trữ dữ liệu trong PostgreSQL, đồng thời cung cấp RESTful API để giao diện React truy xuất dữ liệu và gửi các yêu cầu điều khiển.

Thông qua giao diện Web, người dùng có thể theo dõi tình trạng môi trường, quan sát sự thay đổi của các thông số thông qua biểu đồ, tra cứu lịch sử dữ liệu và thực hiện các thao tác bật hoặc tắt thiết bị từ xa.

Đặc biệt, hệ thống hỗ trợ giao tiếp hai chiều. Thiết bị không chỉ gửi dữ liệu cảm biến lên hệ thống mà còn có thể tiếp nhận lệnh từ Backend và gửi phản hồi trạng thái thực tế sau khi thực hiện. Điều này giúp đảm bảo trạng thái hiển thị trên giao diện phản ánh đúng trạng thái hoạt động của thiết bị.

Việc thực hiện đề tài giúp nhóm tiếp cận đầy đủ các thành phần cơ bản của một hệ thống IoT, từ thu thập dữ liệu ở tầng thiết bị, truyền thông qua MQTT, xử lý và lưu trữ dữ liệu tại Backend cho đến trực quan hóa và tương tác với người dùng trên Frontend.

Bên cạnh mục tiêu đáp ứng yêu cầu của môn học, hệ thống còn có khả năng mở rộng thành nhiều ứng dụng thực tế như nhà thông minh, nông nghiệp thông minh, giám sát nhà kho, giám sát phòng học và các hệ thống tự động hóa.

Do đó, đề tài vừa giúp nhóm củng cố và vận dụng các kiến thức đã học, vừa tạo nền tảng để tiếp tục nghiên cứu và phát triển các hệ thống IoT có quy mô lớn và mức độ tự động hóa cao hơn trong tương lai.

CHƯƠNG 2. THIẾT KẾ TỔNG QUAN

I. Phân tích lớp thực thể

1. Phân tích các lớp thực thể

Hệ thống IoT gồm 5 lớp thực thể chính: User, Device, Sensor, SensorData và Action. Các lớp thực thể được xây dựng nhằm quản lý người dùng, thiết bị, cảm biến, dữ liệu cảm biến và lịch sử thao tác điều khiển trong hệ thống.

1.1 Lớp User

Lớp **User** đại diện cho người dùng sử dụng hệ thống IoT. Lớp này chịu trách nhiệm lưu trữ thông tin tài khoản, thông tin cá nhân và vai trò của người dùng trong hệ thống.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	int	Mã định danh duy nhất của người dùng
name	String	Họ và tên người dùng
email	String	Địa chỉ email
password	String	Mật khẩu đăng nhập
username	String	Tên đăng nhập

phone	String	Số điện thoại
avatar	String	Đường dẫn ảnh đại diện
github	String	Liên kết tài khoản GitHub
figma	String	Liên kết tài khoản Figma
role	String	Vai trò của người dùng, ví dụ ADMIN, USER

Bảng 2.1 Các thuộc tính của lớp User

Lớp User được sử dụng trong các chức năng đăng nhập, quản lý thông tin cá nhân và phân quyền người dùng.

1.2. Lớp Device

Lớp Device đại diện cho các thiết bị IoT được quản lý trong hệ thống. Các thiết bị có khả năng nhận lệnh điều khiển từ hệ thống, chẳng hạn như bật hoặc tắt đèn LED.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	int	Mã định danh duy nhất của thiết bị

name	String	Tên thiết bị
type	String	Loại thiết bị
status	String	Trạng thái hiện tại của thiết bị
mqttTopic	String	MQTT Topic dùng để giao tiếp với thiết bị
createdAt	DateTime	Thời gian tạo thiết bị
updatedAt	DateTime	Thời gian cập nhật thiết bị

Bảng 2.2 Các thuộc tính của lớp Device

Lớp Device phục vụ các chức năng quản lý và điều khiển thiết bị trong hệ thống.

1.3. Lớp Sensor

Lớp Sensor đại diện cho các cảm biến được sử dụng để thu thập thông tin từ môi trường.

Ví dụ:

- Cảm biến nhiệt độ.
- Cảm biến độ ẩm.
- Cảm biến ánh sáng.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	int	Mã định danh duy nhất của cảm biến
name	String	Tên cảm biến
type	String	Loại cảm biến
unit	String	Đơn vị đo
status	String	Trạng thái hoạt động của cảm biến
mqttTopic	String	MQTT Topic dùng để nhận dữ liệu
createdAt	DateTime	Thời gian tạo cảm biến
updatedAt	DateTime	Thời gian cập nhật cảm biến

Bảng 2.3 Thuộc tính của lớp Sensor

Lớp Sensor chỉ lưu thông tin mô tả và trạng thái của cảm biến. Các giá trị đo thực tế được lưu trữ trong lớp SensorData.

1.4. Lớp SensorData

Lớp `SensorData` đại diện cho dữ liệu được thu thập từ cảm biến theo từng thời điểm. Đây là dữ liệu được sử dụng để theo dõi môi trường và hiển thị các biểu đồ trên Dashboard.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	int	Mã định danh của bản ghi dữ liệu
value	double	Giá trị đo được
time	DateTime	Thời điểm ghi nhận dữ liệu
sensorId	int	Mã của cảm biến tạo ra dữ liệu

Bảng 2.4 Thuộc tính của lớp `SensorData`

Ví dụ, cảm biến nhiệt độ có thể tạo ra các dữ liệu:

- 10:00 → 26.1°C
- 10:01 → 26.4°C
- 10:02 → 26.7°C
- 10:03 → 27.0°C

Các dữ liệu này được sử dụng để xây dựng biểu đồ nhiệt độ theo thời gian trên Dashboard.

1.5. Lớp `Action`

Lớp `Action` đại diện cho một thao tác điều khiển thiết bị được thực hiện bởi người dùng. Lớp này được sử dụng để lưu lại lịch sử điều khiển thiết bị.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	int	Mã định danh của thao tác
action	String	Hành động được thực hiện, ví dụ TURN_ON, TURN_OFF
status	String	Kết quả thực hiện, ví dụ SUCCESS, FAILED
time	DateTime	Thời điểm thực hiện thao tác
userId	int	Mã người dùng thực hiện thao tác
deviceId	int	Mã thiết bị được điều khiển

Bảng 2.5 Thuộc tính của lớp Action

Ví dụ một bản ghi Action:

- User: admin
- Device: LED
- Action: TURN_ON
- Status: SUCCESS
- Time: 19/08/2026 10:30:15

Thông tin này được sử dụng trong chức năng Device Control History.

2. Phân tích mối quan hệ giữa các lớp thực thể

Sau khi xác định các lớp thực thể, hệ thống được xây dựng với các mối quan hệ giữa chúng như sau.

2.1. Quan hệ giữa User và Action

Một User có thể thực hiện nhiều thao tác điều khiển thiết bị. Tuy nhiên, mỗi Action chỉ được thực hiện bởi một User.

Do đó, quan hệ giữa User và Action là:

1-N (One-to-Many)

2.2. Quan hệ giữa Device và Action

Một Device có thể được điều khiển nhiều lần trong quá trình hoạt động của hệ thống. Mỗi Action chỉ tác động đến một Device.

Do đó, quan hệ giữa Device và Action là:

1-N (One-to-Many)

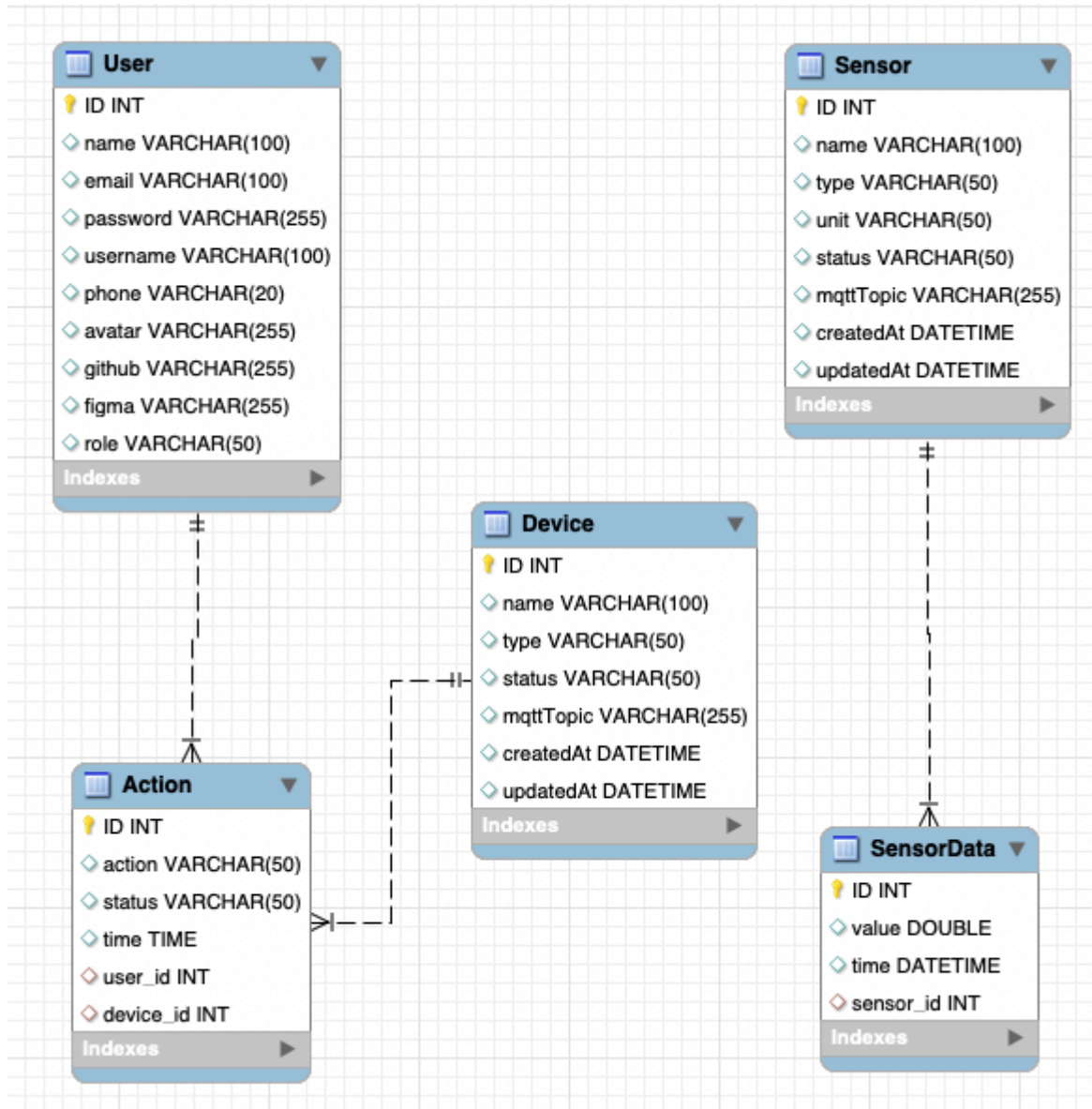
2.3. Quan hệ giữa Sensor và SensorData

Một Sensor có thể tạo ra nhiều bản ghi dữ liệu trong quá trình hoạt động. Ngược lại, mỗi SensorData chỉ thuộc về một Sensor.

Do đó, quan hệ giữa Sensor và SensorData là:

1-N (One-to-Many)

II. Sơ đồ ERD



Hình 2.1 Sơ đồ ERD

III. THIẾT KẾ TỔNG QUAN CÁC THÀNH PHẦN CỦA HỆ THỐNG

Hệ thống IoT giám sát môi trường và điều khiển thiết bị từ xa được xây dựng gồm bốn thành phần chính: phần cứng, Backend, Frontend và cơ sở dữ liệu. Các thành phần được kết nối với nhau thông qua mạng Internet, giao thức MQTT và RESTful API nhằm đảm bảo quá trình thu thập dữ liệu, lưu trữ, hiển thị và điều khiển thiết bị được thực hiện một cách thống nhất.

Kiến trúc tổng quát của hệ thống có thể được mô tả như sau:

Cảm biến → ESP32 → MQTT Broker → Spring Boot → PostgreSQL

Đối với phía người dùng:

PostgreSQL ↔ Spring Boot ↔ React ↔ Người dùng

Khi người dùng thực hiện điều khiển thiết bị, luồng xử lý được thực hiện theo chiều ngược lại:

Người dùng → React → Spring Boot → MQTT Broker → ESP32 → Thiết bị

Sau khi thiết bị thực hiện lệnh, ESP32 gửi trạng thái thực tế trở lại hệ thống để Backend cập nhật trạng thái và lưu lịch sử điều khiển.

3.1. Tổng quan phần cứng

Phần cứng là thành phần trực tiếp tương tác với môi trường vật lý, có nhiệm vụ thu thập dữ liệu cảm biến và thực hiện các lệnh điều khiển từ hệ thống.

Trong hệ thống, ESP32 được sử dụng làm vi điều khiển trung tâm. ESP32 kết nối với các cảm biến và các thiết bị đầu ra, đồng thời sử dụng Wi-Fi để giao tiếp với MQTT Broker.

Hệ thống sử dụng cảm biến DHT11 để thu thập nhiệt độ và độ ẩm của môi trường. Cảm biến BH1750 được sử dụng để đo cường độ ánh sáng. Dữ liệu từ các cảm biến được ESP32 đọc theo chu kỳ và tổng hợp thành các bản tin dữ liệu.

Ngoài các cảm biến, hệ thống sử dụng ba đèn LED làm thiết bị đầu ra để mô phỏng các thiết bị điện có khả năng điều khiển từ xa. ESP32 thực hiện bật hoặc tắt các LED thông qua các chân GPIO dựa trên lệnh nhận được từ hệ thống.

Về kết nối phần cứng, DHT11 được kết nối với ESP32 thông qua chân dữ liệu, trong khi BH1750 sử dụng giao tiếp I2C để truyền dữ liệu. Các LED được kết nối trực tiếp với các chân GPIO của ESP32 thông qua điện trở phù hợp.

Sau khi thu thập dữ liệu, ESP32 đóng gói các giá trị nhiệt độ, độ ẩm và ánh sáng thành dữ liệu có cấu trúc và gửi lên MQTT Broker. Ngược lại, ESP32 cũng đăng ký nhận các lệnh điều khiển từ MQTT Broker để thực hiện thay đổi trạng thái thiết bị.

Như vậy, ESP32 đóng vai trò trung gian giữa môi trường thực tế và hệ thống phần mềm, vừa thực hiện thu thập dữ liệu, vừa thực hiện các lệnh điều khiển thiết bị.

3.2. Tổng quan kết nối MQTT

MQTT được sử dụng làm giao thức truyền thông chính giữa phần cứng và Backend.

MQTT hoạt động theo mô hình Publish/Subscribe, trong đó các thành phần không cần giao tiếp trực tiếp với nhau mà trao đổi dữ liệu thông qua một MQTT Broker.

Đối với quá trình thu thập dữ liệu, ESP32 đóng vai trò Publisher và gửi dữ liệu cảm biến lên MQTT Broker. Spring Boot đăng ký nhận các bản tin tương ứng với vai trò Subscriber.

Luồng dữ liệu được mô tả như sau:

Cảm biến → ESP32 → MQTT Broker → Spring Boot

Đối với quá trình điều khiển thiết bị, Spring Boot đóng vai trò Publisher và gửi lệnh điều khiển đến MQTT Broker. ESP32 đăng ký nhận lệnh với vai trò Subscriber.

Luồng điều khiển được mô tả như sau:

Spring Boot → MQTT Broker → ESP32 → Thiết bị

Sau khi thực hiện lệnh, ESP32 gửi trạng thái thực tế của thiết bị trở lại hệ thống:

Thiết bị → ESP32 → MQTT Broker → Spring Boot

Việc sử dụng MQTT giúp quá trình truyền dữ liệu giữa thiết bị IoT và Backend diễn ra linh hoạt, đồng thời giảm sự phụ thuộc trực tiếp giữa hai thành phần.

3.3. Tổng quan Backend

Backend của hệ thống được xây dựng bằng Java Spring Boot và đóng vai trò trung tâm trong quá trình xử lý dữ liệu.

Spring Boot có nhiệm vụ kết nối với MQTT Broker để tiếp nhận dữ liệu cảm biến từ ESP32. Khi nhận được dữ liệu, Backend tiến hành kiểm tra, xử lý và lưu dữ liệu vào cơ sở dữ liệu PostgreSQL.

Ngoài ra, Backend cung cấp các RESTful API để Frontend có thể truy xuất dữ liệu của hệ thống. Thông qua các API này, React có thể lấy dữ liệu cảm biến, thông tin thiết bị, trạng thái thiết bị và lịch sử điều khiển để hiển thị cho người dùng.

Đối với chức năng điều khiển thiết bị, Spring Boot tiếp nhận yêu cầu từ Frontend, xác định thiết bị cần điều khiển và gửi lệnh tương ứng đến MQTT Broker. ESP32 nhận lệnh, thực hiện thao tác và gửi kết quả trở lại Backend.

Có thể mô tả vai trò của Backend trong hệ thống như sau:

React ↔ Spring Boot ↔ PostgreSQL

và:

Spring Boot ↔ MQTT Broker ↔ ESP32

Như vậy, Spring Boot đóng vai trò cầu nối giữa Frontend, cơ sở dữ liệu và hệ thống phần cứng IoT.

3.4. Tổng quan Frontend

Frontend của hệ thống được xây dựng bằng React nhằm cung cấp giao diện Web cho người dùng.

React chịu trách nhiệm hiển thị dữ liệu mà Backend cung cấp thông qua RESTful API. Người dùng không giao tiếp trực tiếp với ESP32 hoặc cơ sở dữ liệu mà thực hiện mọi thao tác thông qua giao diện Web.

Các chức năng chính trên giao diện gồm:

- Theo dõi nhiệt độ hiện tại.
- Theo dõi độ ẩm hiện tại.
- Theo dõi cường độ ánh sáng.
- Xem biểu đồ dữ liệu cảm biến theo thời gian.
- Xem lịch sử dữ liệu cảm biến.
- Theo dõi trạng thái thiết bị.
- Bật hoặc tắt thiết bị.
- Xem lịch sử điều khiển thiết bị.
- Xem và cập nhật thông tin cá nhân.

Khi cần hiển thị dữ liệu, React gửi yêu cầu đến Spring Boot. Backend truy vấn dữ liệu trong PostgreSQL và trả kết quả về Frontend dưới dạng dữ liệu JSON.

Đối với thao tác điều khiển, người dùng thực hiện bật hoặc tắt thiết bị trên giao diện. React gửi yêu cầu đến Backend, sau đó Backend truyền lệnh đến ESP32 thông qua MQTT.

Vì vậy, Frontend đóng vai trò là lớp giao tiếp trực tiếp giữa người dùng và toàn bộ hệ thống IoT.

3.5. Tổng quan cơ sở dữ liệu

Hệ thống sử dụng PostgreSQL để lưu trữ dữ liệu lâu dài.

Cơ sở dữ liệu có nhiệm vụ lưu các thông tin được tạo ra trong quá trình hệ thống hoạt động, bao gồm:

- Thông tin người dùng.
- Thông tin thiết bị.

- Thông tin cảm biến.
- Dữ liệu cảm biến.
- Lịch sử điều khiển thiết bị.

Các bảng chính trong hệ thống gồm:

- User
- Device
- Sensor
- SensorData
- Action

Trong đó, User lưu thông tin người sử dụng hệ thống; Device lưu thông tin và trạng thái thiết bị; Sensor lưu thông tin của các cảm biến; SensorData lưu dữ liệu cảm biến theo từng thời điểm; Action lưu lại lịch sử điều khiển thiết bị.

Spring Boot là thành phần trực tiếp giao tiếp với PostgreSQL. Frontend và ESP32 không truy cập trực tiếp cơ sở dữ liệu.

Luồng truy cập cơ sở dữ liệu được thực hiện như sau:

React → Spring Boot → PostgreSQL

hoặc đối với dữ liệu từ cảm biến:

ESP32 → MQTT Broker → Spring Boot → PostgreSQL

Việc sử dụng cơ sở dữ liệu giúp hệ thống không chỉ hiển thị dữ liệu hiện tại mà còn có thể lưu lại dữ liệu trong quá khứ để phục vụ việc tra cứu, thống kê và hiển thị biểu đồ.

3.6. Tổng quan luồng hoạt động của hệ thống

Toàn bộ hệ thống có thể được chia thành hai luồng hoạt động chính là luồng giám sát dữ liệu và luồng điều khiển thiết bị.

Đối với chức năng giám sát, dữ liệu được truyền theo luồng:

**Cảm biến → ESP32 → MQTT Broker → Spring Boot → PostgreSQL → React
→ Người dùng**

Các cảm biến thu thập dữ liệu từ môi trường. ESP32 đọc dữ liệu và gửi lên MQTT Broker. Spring Boot tiếp nhận dữ liệu, xử lý và lưu vào PostgreSQL. React sau đó gọi API để lấy dữ liệu và hiển thị cho người dùng.

Đối với chức năng điều khiển, dữ liệu được truyền theo luồng:

Người dùng → React → Spring Boot → MQTT Broker → ESP32 → Thiết bị

Sau khi thiết bị thực hiện lệnh, trạng thái được phản hồi theo chiều ngược lại:

Thiết bị → ESP32 → MQTT Broker → Spring Boot → PostgreSQL → React

Nhờ cơ chế này, hệ thống có khả năng giao tiếp hai chiều, vừa theo dõi dữ liệu môi trường, vừa cho phép người dùng điều khiển thiết bị từ xa.

3.7. Kết luận

Hệ thống được thiết kế theo mô hình gồm nhiều thành phần độc lập nhưng có sự liên kết với nhau. ESP32 chịu trách nhiệm thu thập dữ liệu và điều khiển thiết bị; MQTT Broker chịu trách nhiệm truyền bản tin giữa thiết bị và Backend; Spring Boot chịu trách nhiệm xử lý nghiệp vụ và cung cấp API; PostgreSQL chịu trách nhiệm lưu trữ dữ liệu; React cung cấp giao diện để người dùng theo dõi và điều khiển hệ thống.

Việc phân chia rõ chức năng của từng thành phần giúp hệ thống có cấu trúc dễ quản lý, dễ bảo trì và thuận tiện cho việc mở rộng trong tương lai. Hệ thống có thể bổ sung thêm nhiều loại cảm biến, thiết bị điều khiển hoặc các chức năng tự động hóa mà không cần thay đổi toàn bộ kiến trúc hiện tại.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

I. Xác thực và phân quyền

1. Đăng nhập

a) Mô tả tính năng

Cho phép người dùng đăng nhập vào hệ thống bằng username và password để xác thực danh tính và truy cập các chức năng được cấp quyền.

b) Mô tả chi tiết

Luồng xử lý:

1. Người dùng truy cập trang Login của hệ thống.
2. Người dùng nhập username/email và password.
3. Người dùng nhấn nút Login.
4. Frontend kiểm tra dữ liệu đầu vào và gửi yêu cầu đăng nhập đến Backend thông qua RESTful API.
5. Backend tiếp nhận yêu cầu và tìm kiếm tài khoản người dùng trong Database.
6. Database thực hiện truy vấn thông tin tài khoản dựa trên username/email.
7. Database trả thông tin tài khoản về Backend.
8. Backend kiểm tra mật khẩu người dùng cung cấp với mật khẩu đã được lưu trong hệ thống.
9. Nếu thông tin đăng nhập hợp lệ, Backend tạo JWT/Access Token cho người dùng.
10. Backend trả kết quả đăng nhập thành công và token về Frontend.
11. Frontend lưu token và chuyển người dùng đến DashboardPage.
12. DashboardPage hiển thị giao diện và các chức năng mà người dùng được phép sử dụng.

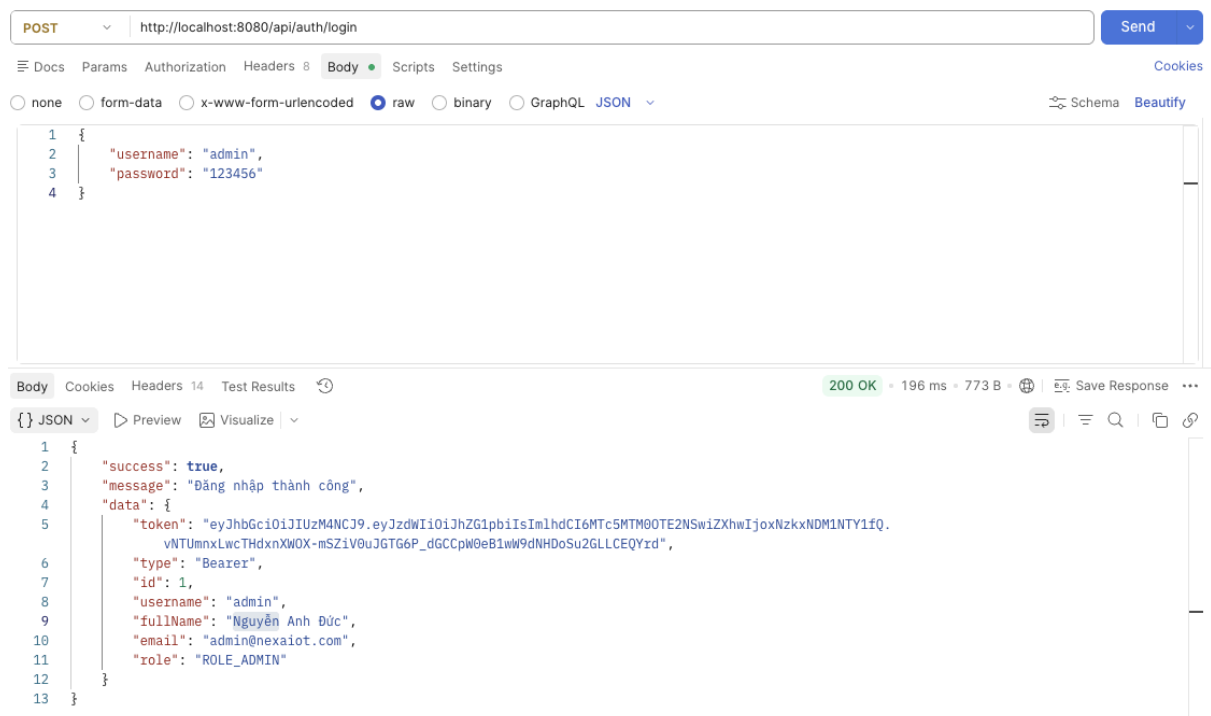
Trường hợp đăng nhập thất bại:

- Nếu tài khoản không tồn tại hoặc mật khẩu không chính xác, Backend trả về lỗi xác thực.
- Frontend hiển thị thông báo “Username hoặc password không chính xác” và yêu cầu người dùng nhập lại.

Các thông tin được sử dụng gồm:

- Username/Email: Tài khoản dùng để đăng nhập.
- Password: Mật khẩu của tài khoản.
- Access Token/JWT: Token được Backend cấp sau khi đăng nhập thành công.
- Kết quả đăng nhập: Thành công hoặc thất bại.
- Thông báo: Thông báo kết quả đăng nhập cho người dùng.

API đề xuất: POST api/auth/login



Hình 3.1 API Login

Thành phần	Nội dung
Method	POST
Endpoint	/api/auth/login
Mô tả	Xác thực người dùng bằng username/email và password, sau đó cấp JWT/Access Token nếu thông tin đăng nhập hợp lệ.
Authentication	Không yêu cầu
Authorization	Không yêu cầu

Bảng 3.1 Thông tin API đăng nhập

Parameter	Kiểu dữ liệu	Bắt buộc	Vị trí	Mô tả
username	String	Có	Body	Username hoặc email dùng để đăng nhập
password	String	Có	Body	Mật khẩu của tài khoản

Bảng 3.2 Tham số Request của API đăng nhập

Field	Kiểu dữ liệu	Mô tả
message	String	Thông báo kết quả đăng nhập
accessToken	String	JWT/Access Token được cấp cho người dùng
user.id	int	ID người dùng
user.username	String	Username của người dùng
user.name	String	Tên người dùng
user.role	String	Vai trò của người dùng

Bảng 3.3 Các trường Response đăng nhập thành công

Các mã phản hồi

HTTP Status	Ý nghĩa
200 OK	Đăng nhập thành công

400 Bad Request	Dữ liệu username/password không hợp lệ hoặc bị thiếu
401 Unauthorized	Username/email hoặc password không chính xác
500 Internal Server Error	Lỗi xảy ra phía Backend/Server

Bảng 3.4 Các mã phản hồi của API đăng nhập

c) **Giao diện người dùng**

Đăng nhập

Đăng nhập để quản lý hệ thống IoT

* Tên đăng nhập

* Mật khẩu

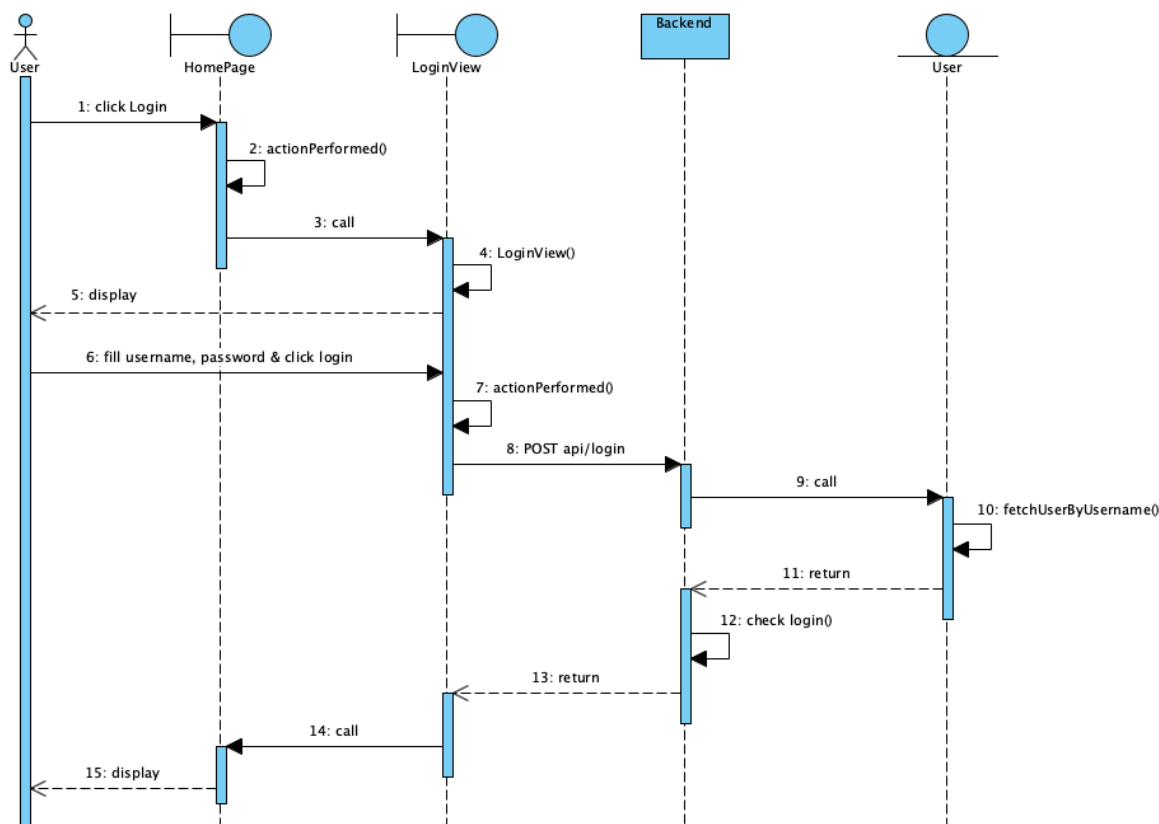
Đăng nhập

Demo: admin / 123456

Chưa có tài khoản? [Đăng ký ngay](#)

Hình 3.2 Giao diện Login

d) **Sequence Diagram**



Hình 3.3 Sequence Diagram Login

Mô tả tuần tự:

1. Người dùng thao tác chọn chức năng đăng nhập trên giao diện trang chủ.
2. Hệ thống ghi nhận sự kiện tương tác vừa diễn ra trên trang chủ.
3. Trang chủ thực hiện lệnh gọi để chuyển hướng sang màn hình đăng nhập.
4. Màn hình giao diện đăng nhập bắt đầu được hệ thống khởi tạo.
5. Giao diện đăng nhập xuất hiện và hiển thị trên màn hình để người dùng tương tác.
6. Người dùng tiến hành điền thông tin tài khoản (bao gồm tên đăng nhập, mật khẩu) và nhấn nút xác nhận đăng nhập.
7. Giao diện đăng nhập ghi nhận sự kiện và thu thập dữ liệu do người dùng vừa nhập.
8. Giao diện đóng gói thông tin và gửi yêu cầu xác thực tài khoản đến máy chủ (Backend).

9. Máy chủ tiếp nhận yêu cầu và gửi lệnh truy vấn tới bộ phận quản lý dữ liệu (Thực thể User / Cơ sở dữ liệu).
10. Cơ sở dữ liệu thực thi quá trình tìm kiếm thông tin tài khoản dựa trên tên đăng nhập vừa được cung cấp.
11. Cơ sở dữ liệu trả về thông tin của người dùng tương ứng cho máy chủ.
12. Máy chủ tiến hành kiểm tra, đối chiếu dữ liệu (như kiểm tra mật khẩu) để xác thực tính hợp lệ của phiên đăng nhập.
13. Máy chủ hoàn tất quá trình kiểm tra và trả kết quả phản hồi về cho giao diện đăng nhập.
14. Khi nhận được kết quả hợp lệ, giao diện đăng nhập thực hiện lệnh điều hướng người dùng trở lại giao diện trang chủ.
15. Giao diện trang chủ cập nhật trạng thái và hiển thị lại lên màn hình cho người dùng.

2. Đăng ký tài khoản

a) Mô tả tính năng

Cho phép người dùng tạo tài khoản mới trên hệ thống bằng cách cung cấp các thông tin đăng nhập (username, password) để hệ thống lưu trữ, từ đó người dùng có thể sử dụng để đăng nhập và truy cập các chức năng của ứng dụng.

b) Mô tả chi tiết

Luồng xử lý:

1. Người dùng đang ở trang chủ (HomePage) và nhấn chọn chức năng Đăng ký tài khoản.
2. Hệ thống xử lý sự kiện và hiển thị màn hình giao diện Đăng ký (RegisterView) cho người dùng.
3. Người dùng nhập các thông tin bao gồm: username, password, confirmPassword và nhấn nút xác nhận (OK).
4. Frontend (RegisterView) thực hiện kiểm tra tính hợp lệ của dữ liệu đầu vào (validate).

5. Nếu dữ liệu hợp lệ, Frontend gửi yêu cầu tạo tài khoản mới đến Backend thông qua RESTful API (phương thức POST api/register).
6. Backend tiếp nhận yêu cầu và gọi xuống Database (Thực thể User) để kiểm tra xem tên đăng nhập này đã tồn tại trong hệ thống hay chưa (checkExistsByUsername()).
7. Database thực hiện truy vấn và trả kết quả kiểm tra về cho Backend.
8. Nếu username chưa tồn tại, Backend thực hiện lệnh lưu thông tin tài khoản người dùng mới vào Database (save()).
9. Database lưu trữ dữ liệu thành công và trả phản hồi về Backend.
10. Backend trả kết quả đăng ký thành công về cho Frontend (RegisterView).
11. Frontend hiển thị thông báo "Đăng ký thành công" cho người dùng.
12. Người dùng nhấn nút "OK" để xác nhận đóng thông báo.
13. Frontend tự động chuyển hướng người dùng quay trở lại trang chủ (HomePage).
14. HomePage hiển thị lại giao diện ban đầu.

Trường hợp đăng ký thất bại:

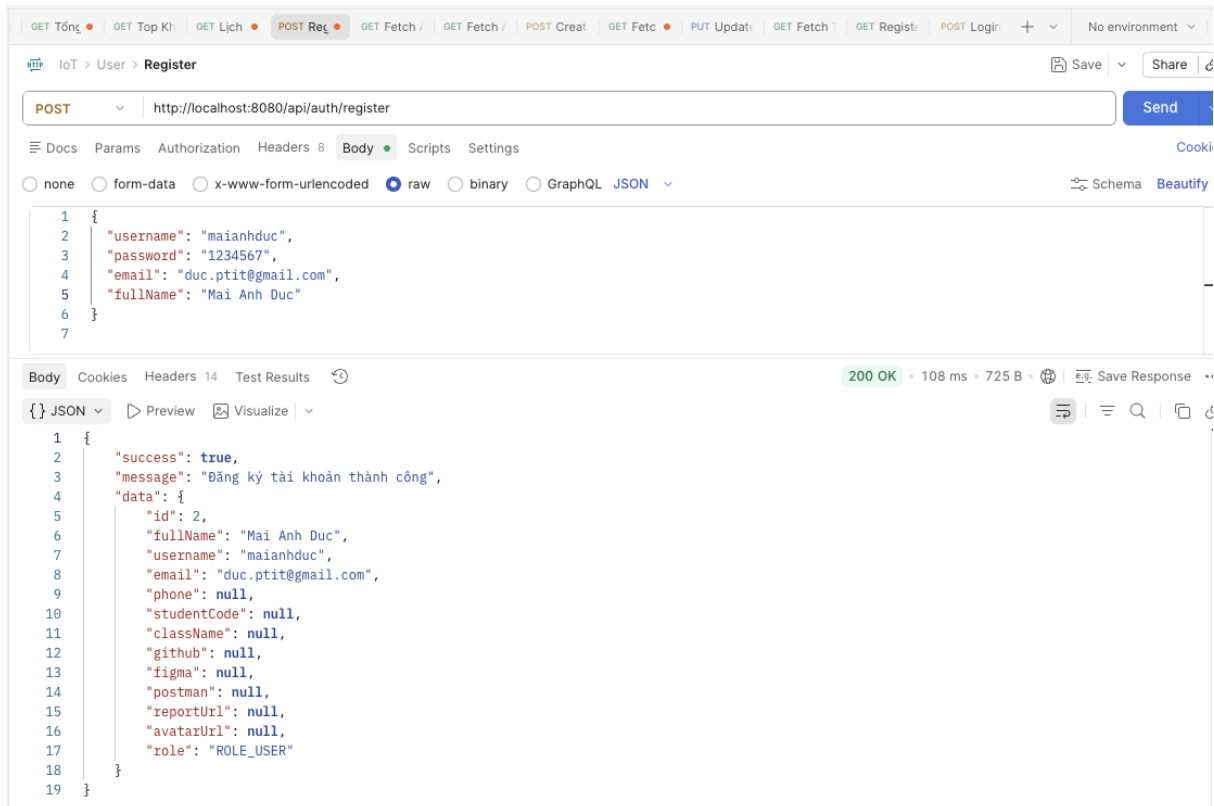
- Nếu dữ liệu đầu vào không hợp lệ (ví dụ: password và confirmPassword không khớp, hoặc thông tin bị bỏ trống), Frontend (ở bước validate) sẽ chặn lại, hiển thị thông báo lỗi và yêu cầu người dùng nhập lại.
- Nếu username mà người dùng nhập đã tồn tại trong hệ thống, Backend sẽ trả về lỗi trùng lặp dữ liệu. Frontend nhận lỗi, hiển thị thông báo “Tên đăng nhập đã tồn tại” và yêu cầu người dùng chọn tên khác.
- Nếu có lỗi trong quá trình lưu dữ liệu xuống Database, Backend trả về lỗi hệ thống và Frontend hiển thị thông báo thử lại sau.

Các thông tin được sử dụng gồm:

- Username: Tên đăng nhập người dùng muốn tạo.
- Password: Mật khẩu bảo mật cho tài khoản.
- Confirm Password: Chuỗi nhập lại mật khẩu để xác minh tính chính xác.

- Kết quả đăng ký: Trạng thái hệ thống trả về (Thành công hoặc thất bại).
- Thông báo: Nội dung hiển thị kết quả hoặc lỗi gửi đến người dùng trong quá trình thao tác.

API đề xuất: POST api/auth/register



Hình 3.4 API Đăng ký tài khoản

c) **Giao diện người dùng**

Đăng ký tài khoản

Tạo tài khoản để sử dụng hệ thống IoT

1

Tài khoản

2

Cá nhân

3

Hoàn thành

*

Tên đăng nhập

Nhập tên đăng nhập

*

Mật khẩu

Tạo mật khẩu

*

Xác nhận mật khẩu

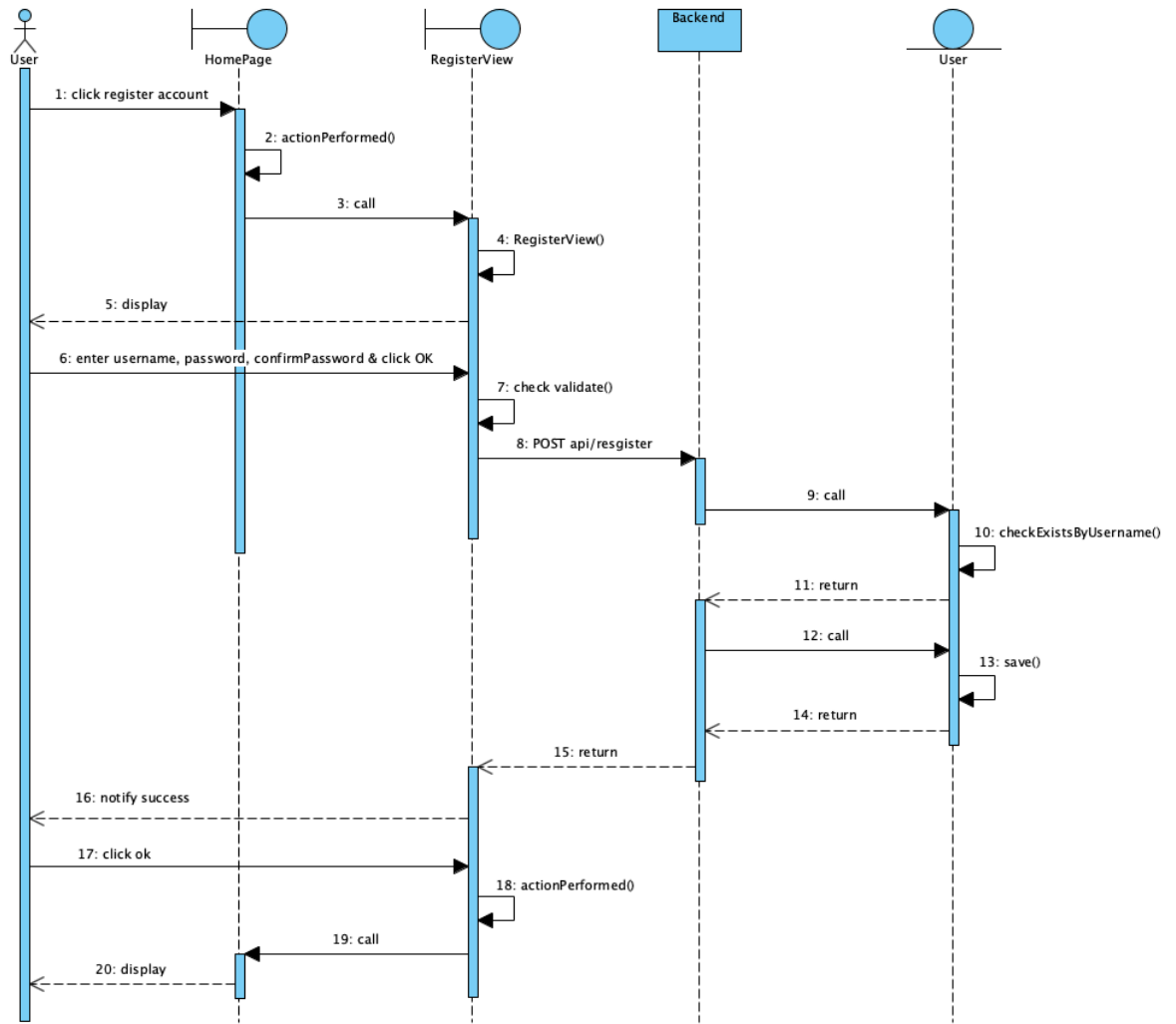
Nhập lại mật khẩu

Tiếp theo →

Đã có tài khoản? [Đăng nhập](#)

Hình 3.5 Giao diện đăng ký tài khoản

d) **Sequence Diagram**



Hình 3.6 Sequence Diagram Đăng ký tài khoản

Mô tả tuần tự:

1. Người dùng thao tác chọn chức năng đăng ký tài khoản trên giao diện trang chủ.
2. Hệ thống ghi nhận sự kiện tương tác vừa diễn ra trên trang chủ.
3. Trang chủ thực hiện lệnh gọi để chuyển hướng sang màn hình đăng ký.
4. Màn hình giao diện đăng ký bắt đầu được hệ thống khởi tạo.
5. Giao diện đăng ký xuất hiện và hiển thị trên màn hình để người dùng tương tác.
6. Người dùng tiến hành điền các thông tin cần thiết bao gồm: tên đăng nhập, mật khẩu, xác nhận mật khẩu và nhấn nút xác nhận (OK).

7. Giao diện đăng ký tự động kiểm tra tính hợp lệ của các dữ liệu mà người dùng vừa nhập (ví dụ: mật khẩu có khớp nhau không, độ dài có đạt yêu cầu không).
8. Sau khi kiểm tra dữ liệu hợp lệ ở phía giao diện, hệ thống đóng gói thông tin và gửi yêu cầu đăng ký tài khoản mới đến máy chủ (Backend).
9. Máy chủ tiếp nhận yêu cầu và gửi lệnh truy vấn tới cơ sở dữ liệu (Thực thể User).
10. Cơ sở dữ liệu tiến hành kiểm tra xem tên đăng nhập mà người dùng vừa nhập đã tồn tại trong hệ thống hay chưa.
11. Cơ sở dữ liệu trả kết quả kiểm tra (tài khoản chưa tồn tại/hợp lệ) về cho máy chủ.
12. Máy chủ tiếp tục gửi lệnh yêu cầu lưu trữ thông tin người dùng mới xuống cơ sở dữ liệu.
13. Cơ sở dữ liệu thực thi quá trình lưu thông tin tài khoản mới vào hệ thống.
14. Cơ sở dữ liệu lưu thành công và trả phản hồi về cho máy chủ.
15. Máy chủ hoàn tất quá trình xử lý và trả kết quả phản hồi đăng ký thành công về cho giao diện đăng ký.
16. Giao diện đăng ký hiển thị thông báo "Đăng ký thành công" trên màn hình cho người dùng.
17. Người dùng tương tác nhấn "OK" để xác nhận và đóng thông báo.
18. Giao diện đăng ký ghi nhận hành động đóng thông báo của người dùng.
19. Giao diện đăng ký thực hiện lệnh điều hướng, đưa người dùng quay trở lại giao diện trang chủ.
20. Giao diện trang chủ được cập nhật và hiển thị lại lên màn hình.

3. Profile

a) Mô tả tính năng

Cho phép người dùng xem và quản lý thông tin cá nhân của tài khoản, bao gồm họ tên, username, email, số điện thoại, ảnh đại diện, GitHub, Figma và

mật khẩu. Người dùng có thể cập nhật các thông tin cá nhân và thay đổi mật khẩu nhằm đảm bảo thông tin tài khoản luôn chính xác, đầy đủ và an toàn.

b) Mô tả chi tiết

Luồng xử lý:

1. Người dùng chọn chức năng Profile trên giao diện hệ thống.
2. Frontend gửi yêu cầu lấy thông tin cá nhân đến Backend thông qua RESTful API.
3. Backend tiếp nhận yêu cầu và xác thực người dùng thông qua JWT/Access Token.
4. Backend lấy thông tin userId từ token và truy vấn thông tin tài khoản trong Database.
5. Database thực hiện truy vấn tìm kiếm thông tin người dùng theo userId.
6. Database trả thông tin tài khoản về Backend.
7. Backend xử lý dữ liệu và trả thông tin Profile về Frontend.
8. Frontend hiển thị thông tin cá nhân của người dùng trên trang Profile.
9. Người dùng có thể xem các thông tin như họ tên, username, email, số điện thoại, ảnh đại diện, GitHub và Figma.
10. Khi muốn thay đổi thông tin, người dùng chọn Edit Profile.
11. Người dùng chỉnh sửa các thông tin cá nhân và nhấn Save.
12. Frontend gửi yêu cầu cập nhật thông tin đến Backend thông qua RESTful API.
13. Backend xác thực người dùng và kiểm tra tính hợp lệ của dữ liệu được gửi lên.
14. Backend yêu cầu Database cập nhật thông tin cá nhân.
15. Database thực hiện cập nhật và trả kết quả về Backend.
16. Backend trả kết quả cập nhật về Frontend.
17. Frontend cập nhật lại thông tin Profile và hiển thị thông báo cập nhật thành công.
18. Khi muốn thay đổi mật khẩu, người dùng chọn chức năng Change Password.

19. Người dùng nhập mật khẩu hiện tại, mật khẩu mới và xác nhận mật khẩu mới.
20. Frontend gửi yêu cầu đổi mật khẩu đến Backend.
21. Backend xác thực người dùng và kiểm tra mật khẩu hiện tại.
22. Backend kiểm tra mật khẩu mới và mật khẩu xác nhận có khớp hay không.
23. Nếu hợp lệ, Backend mã hóa mật khẩu mới trước khi lưu vào Database.
24. Database cập nhật mật khẩu mới và trả kết quả về Backend.
25. Backend trả kết quả đổi mật khẩu về Frontend.
26. Frontend hiển thị thông báo đổi mật khẩu thành công cho người dùng.

Các thông tin được hiển thị gồm:

- **Ảnh đại diện:** Ảnh đại diện của người dùng.
- **Họ tên:** Họ và tên của người dùng.
- **Username:** Tên tài khoản đăng nhập.
- **Email:** Địa chỉ email của người dùng.
- **Số điện thoại:** Số điện thoại liên hệ.
- **GitHub:** Đường dẫn đến tài khoản GitHub.
- **Figma:** Đường dẫn đến tài khoản hoặc trang Figma.
- **Mật khẩu:** Được ẩn và không hiển thị trực tiếp vì lý do bảo mật.

Các thao tác:

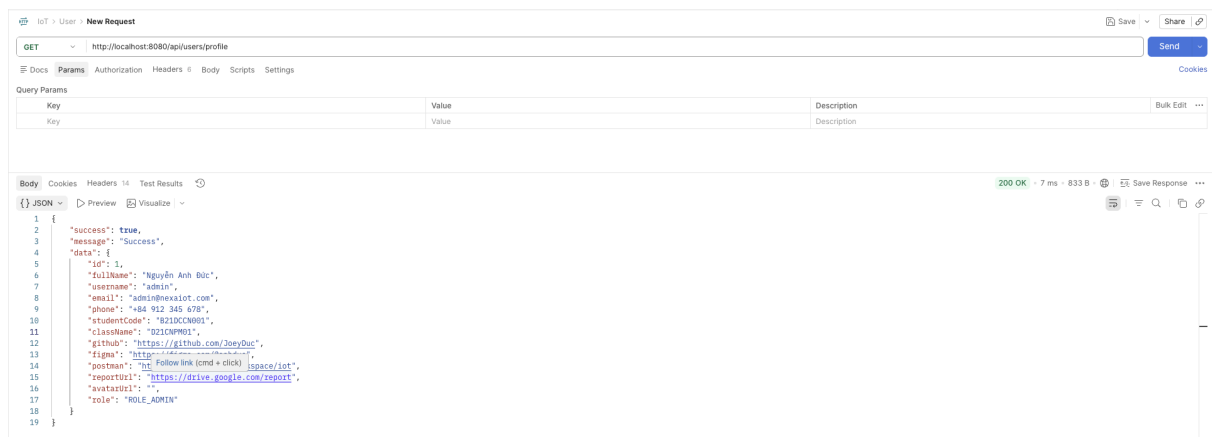
- **View Profile:** Xem thông tin cá nhân.
- **Edit Profile:** Chỉnh sửa thông tin cá nhân.
- **Change Avatar:** Thay đổi ảnh đại diện.
- **Change Password:** Thay đổi mật khẩu.
- **Save:** Lưu các thay đổi.
- **Cancel:** Hủy thay đổi.

Kết quả:

- **Thành công:** Thông tin được cập nhật và giao diện hiển thị dữ liệu mới.
- **Thất bại:** Backend trả về lỗi và Frontend hiển thị thông báo tương ứng, ví dụ email đã tồn tại, dữ liệu không hợp lệ hoặc mật khẩu hiện tại không chính xác.

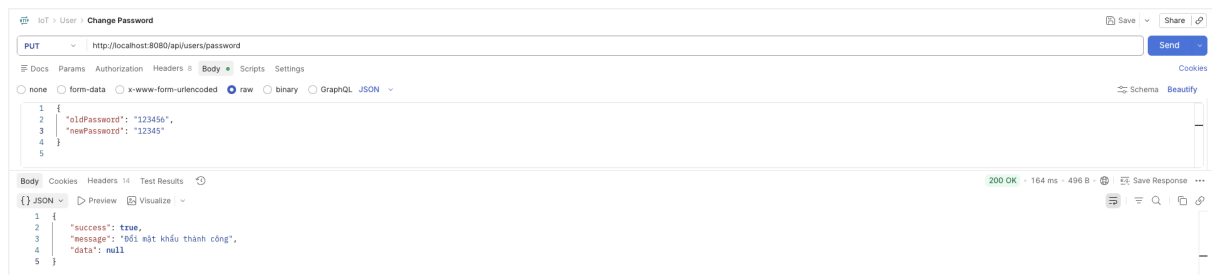
API đề xuất:

GET api/auth/profile



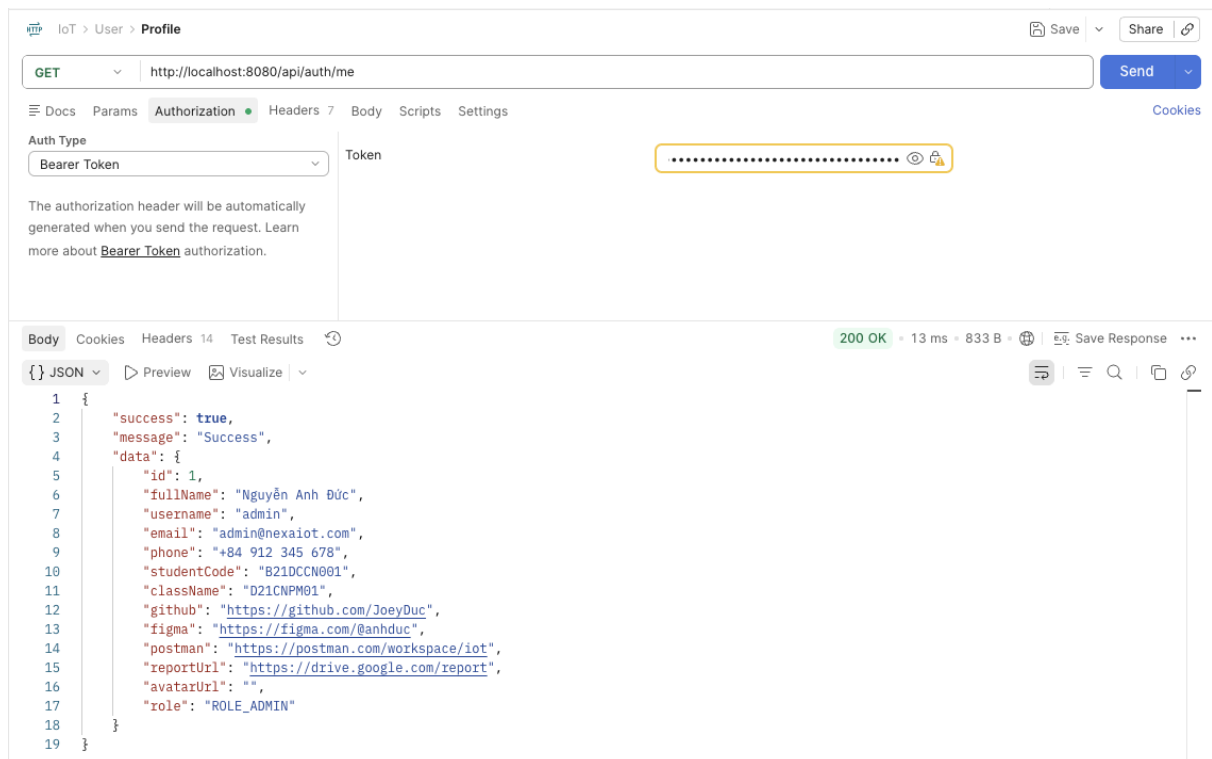
Hình 3.7 API Profile

PUT api/users/password



Hình 3.8 API Change Password

GET api/auth/me



Hình 3.9 API Me

Thành phần	Nội dung
Method	GET
Endpoint	/api/auth/profile
Mô tả	Lấy thông tin cá nhân của người dùng đang đăng nhập.
Authentication	Required
Authorization	Người dùng đã đăng nhập

Bảng 3.5 Thông tin API thông tin cá nhân

Request: Không yêu cầu Request Body.

Header: Authorization: Bearer <accessToken>

Parameter	Kiểu dữ liệu	Bắt buộc	Vị trí	Mô tả
Authorization	String	Có	Header	JWT/Access Token của người dùng

Bảng 3.6 Tham số Request của API Profile

Field	Kiểu dữ liệu	Mô tả
id	int	ID người dùng
name	String	Họ và tên
username	String	Tên đăng nhập
email	String	Email
phone	String	Số điện thoại
avatar	String	Đường dẫn ảnh đại diện

github	String	Đường dẫn GitHub
figma	String	Đường dẫn Figma
role	String	Vai trò của người dùng

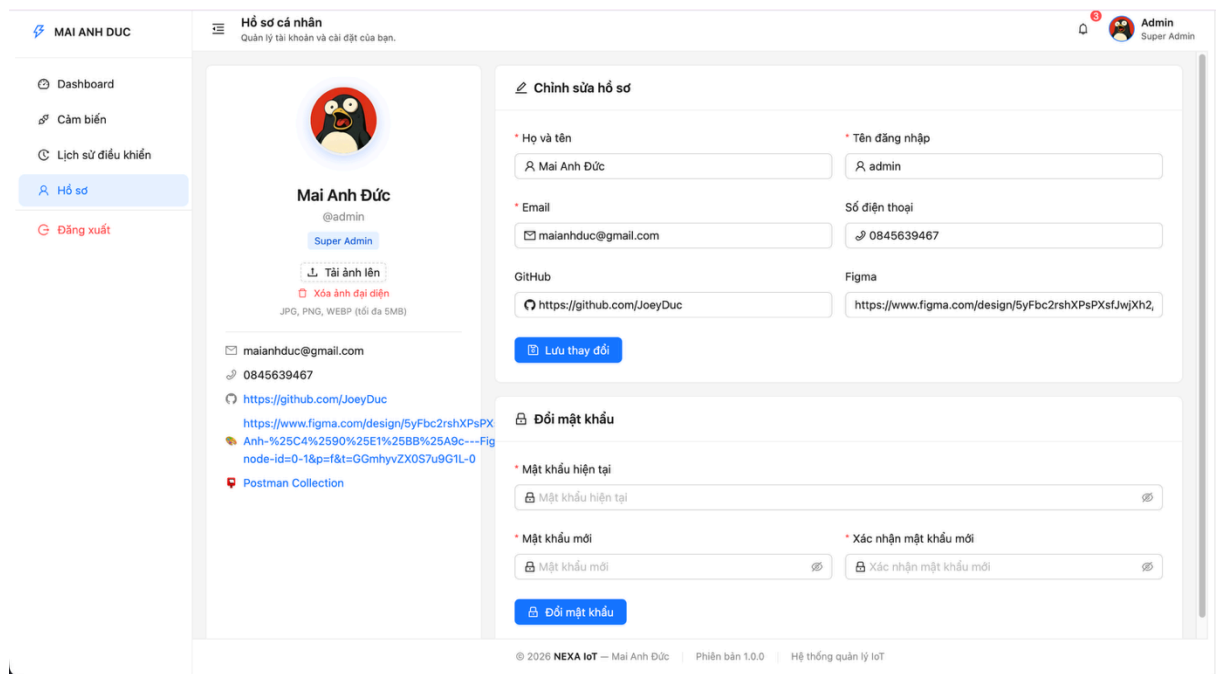
Bảng 3.7 Các trường Response API Profile thành công

Các mã phản hồi

HTTP Status	Ý nghĩa
200 OK	Lấy thông tin thành công
401 Unauthorized	Token không tồn tại hoặc không hợp lệ
404 Not Found	Không tìm thấy người dùng
500 Internal Server Error	Lỗi phía Server

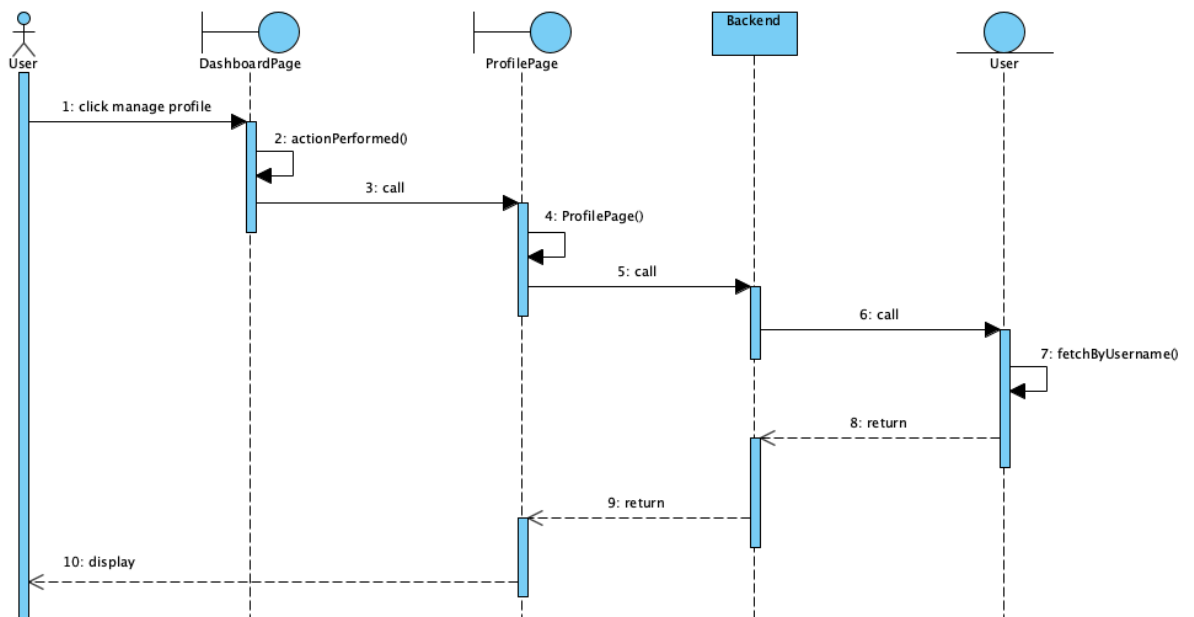
Bảng 3.8 Các mã phản hồi của API Profile

c) Giao diện người dùng



Hình 3.10 Giao diện Profile

d) Sequence Diagram



Hình 3.11 Sequence Diagram Profile

Mô tả tuần tự:

1. Người dùng đang ở giao diện trang tổng quan và nhấn chọn chức năng quản lý hồ sơ cá nhân (manage profile).

2. Giao diện trang tổng quan ghi nhận sự kiện click của người dùng.
3. Trang tổng quan thực hiện lệnh gọi để chuyển hướng người dùng sang màn hình giao diện Hồ sơ cá nhân (ProfilePage).
4. Giao diện Hồ sơ cá nhân bắt đầu được hệ thống khởi tạo.
5. Ngay khi khởi tạo, giao diện này tự động gửi một yêu cầu (API) đến máy chủ (Backend) để lấy dữ liệu thông tin người dùng.
6. Máy chủ tiếp nhận yêu cầu và gọi xuống Cơ sở dữ liệu (Thực thể User) để yêu cầu trích xuất thông tin.
7. Cơ sở dữ liệu thực thi truy vấn tìm kiếm hồ sơ người dùng dựa trên tên đăng nhập (username) của phiên hoạt động hiện tại.
8. Cơ sở dữ liệu tìm thấy dữ liệu và trả thông tin hồ sơ về cho máy chủ.
9. Máy chủ nhận dữ liệu, xử lý và trả kết quả phản hồi về cho giao diện Hồ sơ cá nhân.
10. Giao diện nhận được dữ liệu và hiển thị đầy đủ thông tin cá nhân lên màn hình cho người dùng theo dõi.

II. Giám sát cảm biến

Chức năng giám sát dữ liệu cảm biến cho phép người dùng theo dõi các thông số môi trường được thu thập trực tiếp từ các cảm biến thông qua ESP8266/ESP32. Hệ thống tập trung giám sát ba thông số chính gồm nhiệt độ, độ ẩm và cường độ ánh sáng.

ESP8266/ESP32 định kỳ đọc dữ liệu từ các cảm biến và gửi dữ liệu lên hệ thống thông qua giao thức MQTT. Backend tiếp nhận dữ liệu, xử lý và lưu trữ vào cơ sở dữ liệu. Frontend sử dụng RESTful API để lấy dữ liệu và hiển thị cho người dùng trên Dashboard.

1. Xem dữ liệu cảm biến

a) Mô tả tính năng

Chức năng **Xem dữ liệu cảm biến** cho phép người dùng theo dõi các thông số môi trường được thu thập từ hệ thống IoT, bao gồm nhiệt độ, độ ẩm và cường độ ánh sáng.

Dữ liệu được thu thập từ các cảm biến thông qua ESP8266/ESP32 và truyền về hệ thống bằng giao thức MQTT. Backend tiếp nhận dữ liệu, kiểm tra tính hợp lệ, xử lý và lưu trữ vào Database. Người dùng có thể xem các giá trị cảm biến mới nhất trên giao diện Dashboard.

Trong quá trình xử lý dữ liệu, Backend đồng thời thực hiện kiểm tra trạng thái của cảm biến và giá trị nhận được để phát hiện các trường hợp bất thường.

Cảnh báo cảm biến mất kết nối

Backend lưu thời gian nhận dữ liệu gần nhất của từng cảm biến. Nếu không nhận được dữ liệu mới trong khoảng thời gian quy định, ví dụ 5 phút, Backend chuyển trạng thái cảm biến từ ONLINE sang OFFLINE và tạo cảnh báo cho người dùng.

Ví dụ: Cảm biến ánh sáng không gửi dữ liệu trong hơn 5 phút.

Cảnh báo giá trị vượt ngưỡng

Backend kiểm tra giá trị cảm biến với các ngưỡng được cấu hình trước.

Ví dụ:

- Nhiệt độ lớn hơn 35°C.
- Độ ẩm lớn hơn 85%.
- Độ ẩm nhỏ hơn 30%.
- Cường độ ánh sáng vượt khỏi ngưỡng được cấu hình.

Nếu giá trị vượt ngưỡng, Backend tạo cảnh báo với mức WARNING.

Ví dụ:

Nhiệt độ vượt ngưỡng cho phép: 38°C.

Cảnh báo dữ liệu không hợp lệ

Backend kiểm tra tính hợp lệ của dữ liệu trước khi lưu vào Database.

Ví dụ:

- Độ ẩm nhỏ hơn 0% hoặc lớn hơn 100%.
- Nhiệt độ có giá trị bất thường ngoài phạm vi đo của cảm biến.
- Dữ liệu bị thiếu trường.
- Dữ liệu không đúng kiểu dữ liệu yêu cầu.

Nếu phát hiện dữ liệu không hợp lệ, Backend không sử dụng giá trị đó làm dữ liệu hiển thị hiện tại và tạo cảnh báo ERROR.

Ví dụ:

Cảm biến độ ẩm gửi dữ liệu không hợp lệ: 125%.

b) Mô tả chi tiết

Luồng xử lý dữ liệu:

1. Cảm biến thực hiện đo các thông số môi trường.
2. ESP8266/ESP32 đọc giá trị từ các cảm biến.
3. ESP8266/ESP32 đóng gói dữ liệu.
4. ESP8266/ESP32 gửi dữ liệu lên MQTT Broker.
5. MQTT Broker chuyển message đến Backend.
6. Backend nhận dữ liệu từ MQTT Broker.
7. Backend kiểm tra định dạng và tính hợp lệ của dữ liệu.

Trường hợp dữ liệu hợp lệ:

8. Backend kiểm tra giá trị cảm biến có vượt ngưỡng được cấu hình hay không.
9. Nếu không vượt ngưỡng, dữ liệu được xử lý bình thường.
10. Nếu vượt ngưỡng, Backend tạo cảnh báo WARNING.
11. Backend lưu dữ liệu cảm biến vào Database.
12. Backend cập nhật thời gian nhận dữ liệu gần nhất của cảm biến.
13. Backend cập nhật trạng thái cảm biến thành ONLINE.

Trường hợp dữ liệu không hợp lệ:

8. Backend xác định dữ liệu không hợp lệ.
9. Backend tạo cảnh báo ERROR.
10. Backend bỏ qua dữ liệu không hợp lệ hoặc lưu riêng phục vụ kiểm tra lỗi.
11. Dữ liệu không hợp lệ không được sử dụng làm giá trị mới nhất trên Dashboard.

Luồng người dùng xem dữ liệu:

1. User truy cập Dashboard.
2. Frontend gửi request đến Backend thông qua RESTful API.
3. Request chứa JWT Token của người dùng.
4. Backend xác thực JWT Token.
5. Backend truy vấn Database để lấy dữ liệu cảm biến mới nhất.
6. Backend lấy trạng thái hiện tại của các cảm biến.
7. Backend lấy các cảnh báo đang hoạt động.
8. Backend trả dữ liệu về Frontend.
9. Frontend hiển thị dữ liệu, trạng thái và cảnh báo trên Dashboard.

Kiểm tra cảm biến mất kết nối

1. Backend định kỳ kiểm tra thời gian nhận dữ liệu gần nhất của từng cảm biến.
2. Backend tính khoảng thời gian từ lần nhận dữ liệu cuối đến thời điểm hiện tại.
3. Nếu thời gian nhỏ hơn hoặc bằng 5 phút, cảm biến được xác định là ONLINE.
4. Nếu thời gian lớn hơn 5 phút, cảm biến được xác định là OFFLINE.
5. Backend cập nhật trạng thái cảm biến thành OFFLINE.
6. Backend tạo cảnh báo mất kết nối.
7. Frontend nhận thông tin cảnh báo và hiển thị trên Dashboard.

Dashboard hiển thị các thông tin sau:

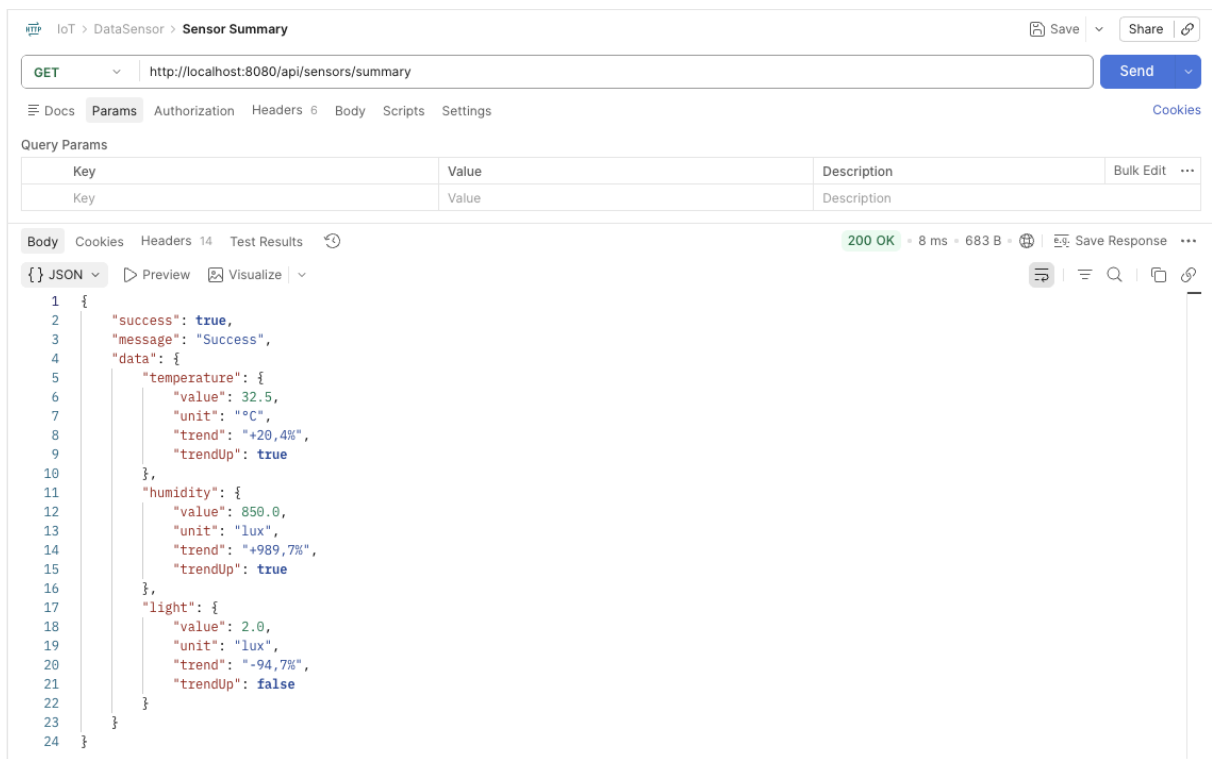
- Tên hoặc loại cảm biến.
- Giá trị đo được.
- Đơn vị đo.
- Thời gian đo.
- Thời gian nhận dữ liệu gần nhất.
- Trạng thái cảm biến: ONLINE hoặc OFFLINE.
- Thời gian cảm biến không gửi dữ liệu.
- Trạng thái giá trị: NORMAL, WARNING hoặc ERROR.
- Nội dung cảnh báo nếu có.

Các loại cảnh báo

Loại	Điều kiện	Mức cảnh báo	Ví dụ
Mất kết nối	Không nhận dữ liệu trong hơn 5 phút	ERROR	Cảm biến ánh sáng đã OFFLINE
Vượt ngưỡng	Giá trị vượt ngưỡng được cấu hình	WARNING	Nhiệt độ 38°C vượt ngưỡng 35°C
Dữ liệu không hợp lệ	Giá trị không nằm trong phạm vi hợp lệ	ERROR	Độ ẩm 125% không hợp lệ

Bảng 3.9 Các loại cảnh báo của hệ thống

API đề xuất: GET api/sensors/summary



Hình 3.12 API Xem dữ liệu cảm biến

Thuộc tính	Mô tả
Endpoint	/api/sensors/summary
Method	GET
Mục đích	Lấy dữ liệu cảm biến mới nhất, trạng thái và cảnh báo.
Authentication	Có – yêu cầu JWT Token.

Content-Type	application/json
Request Body	Không có.
Query Parameters	Không có.
Path Parameters	Không có.

Bảng 3.10 Thông tin API hiển thị dữ liệu của cảm biến

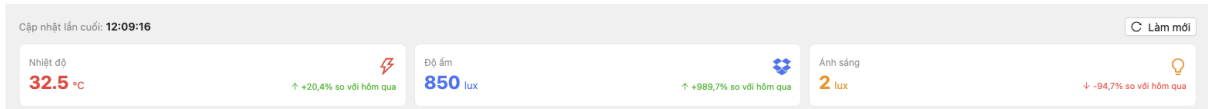
Các mã trạng thái trả về:

Mã	Tên	Mô tả
200	OK	Lấy dữ liệu cảm biến thành công.
401	Unauthorized	Người dùng chưa đăng nhập hoặc JWT Token không hợp lệ/hết hạn.
403	Forbidden	Người dùng đã xác thực nhưng không có quyền truy cập API.
404	Not Found	Không tìm thấy dữ liệu cảm biến.

500	Internal Server Error	Có lỗi xảy ra phía Backend/Server.
-----	-----------------------	------------------------------------

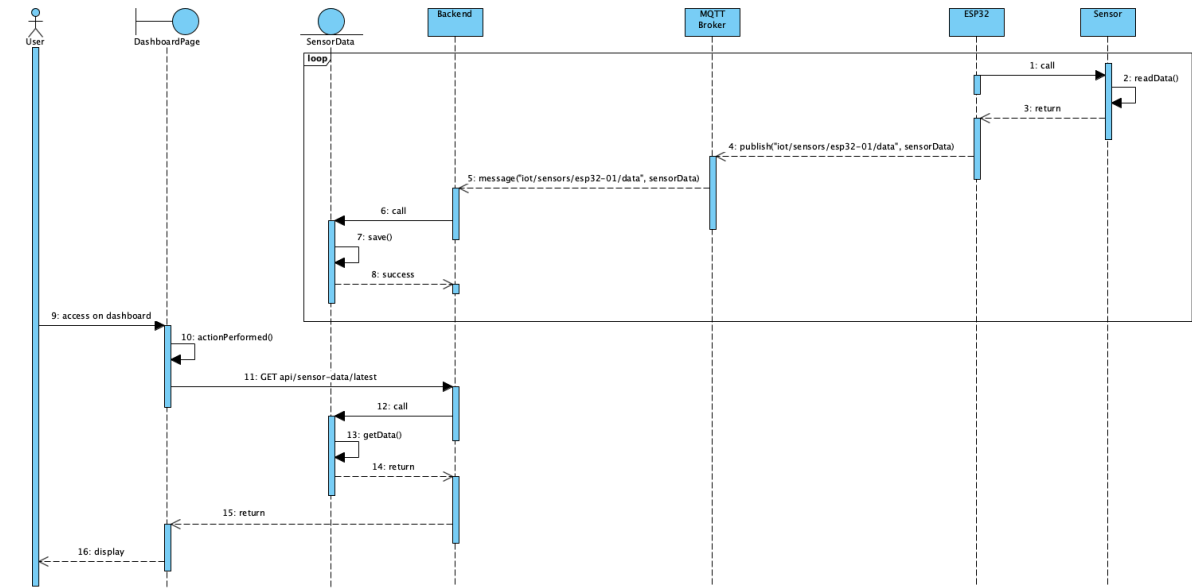
Bảng 3.11 Các mã phản hồi của API đo dữ liệu cảm biến

c) Giao diện người dùng



Hình 3.13 Giao diện hiển thị dữ liệu cảm biến

d) Sequence Diagram



Hình 3.14 Sequence Diagram Hiển thị dữ liệu cảm biến

Mô tả tuần tự scenario:

1. **ESP32** gọi **Sensor** để yêu cầu Sensor đọc dữ liệu.
2. **Sensor** thực hiện readData() để đọc giá trị cảm biến.
 - **Ngoại lệ:** Nếu Sensor không phản hồi hoặc xảy ra lỗi khi đọc dữ liệu, Sensor trả về lỗi cho ESP32 và không tiếp tục gửi dữ liệu.
3. **Sensor** trả dữ liệu về **ESP32** sau khi đọc xong.

4. **ESP32** publish dữ liệu lên **MQTT Broker** thông qua một MQTT topic.
 - **Ngoại lệ:** Nếu ESP32 không kết nối được với MQTT Broker hoặc publish thất bại, dữ liệu không được gửi và ESP32 thực hiện kết nối lại.
5. **MQTT Broker** gửi message đến **Backend** vì Backend đã subscribe topic tương ứng.
 - **Ngoại lệ:** Nếu MQTT Broker không hoạt động hoặc Backend chưa subscribe topic, Backend không nhận được dữ liệu.
6. **Backend** gọi **Database** để lưu dữ liệu cảm biến nhận được.
7. **Database** thực hiện save() và lưu dữ liệu vào hệ thống.
 - **Ngoại lệ:** Nếu Database xảy ra lỗi hoặc không thể lưu dữ liệu, Database trả về lỗi cho Backend.
8. **Database** trả về success cho **Backend** để xác nhận dữ liệu đã được lưu thành công.
9. **User** truy cập Dashboard để xem dữ liệu cảm biến.
10. **DashboardView** thực hiện actionPerformed() để xử lý thao tác của User.
11. **DashboardView** gửi request GET /api/sensor-data/latest đến **Backend** để lấy dữ liệu cảm biến mới nhất.
 - **Ngoại lệ:** Nếu User chưa đăng nhập hoặc JWT không hợp lệ/hết hạn, Backend trả về 401 Unauthorized và Dashboard yêu cầu User đăng nhập lại.
12. **Backend** gọi **Database** để truy vấn dữ liệu.
13. **Database** thực hiện getData() để lấy dữ liệu cảm biến mới nhất.
 - **Ngoại lệ:** Nếu không có dữ liệu cảm biến, Database trả về kết quả rỗng và Backend thông báo không có dữ liệu cho Dashboard.
14. **Database** trả dữ liệu về **Backend** sau khi truy vấn thành công.
15. **Backend** trả dữ liệu về **DashboardView** thông qua API.
 - **Ngoại lệ:** Nếu Backend xảy ra lỗi trong quá trình xử lý hoặc truy vấn, Backend trả về 500 Internal Server Error.
16. **DashboardView** hiển thị dữ liệu lên Dashboard cho **User**.
 - **Ngoại lệ:** Nếu Dashboard không thể nhận hoặc hiển thị dữ liệu, hệ thống thông báo lỗi và yêu cầu User thử tải lại dữ liệu.

2. Xem biểu đồ

a) Mô tả tính năng

Chức năng **Xem biểu đồ** cho phép người dùng theo dõi sự thay đổi của dữ liệu cảm biến theo thời gian thông qua biểu đồ trực quan trên Dashboard.

Người dùng có thể xem **các lần đo gần nhất** của cảm biến nhiệt độ, độ ẩm hoặc ánh sáng. Hệ thống hỗ trợ lựa chọn số lượng dữ liệu cần hiển thị, ví dụ **15 lần đo gần nhất**, giúp người dùng nhanh chóng theo dõi tình trạng môi trường hiện tại và nhận biết xu hướng tăng, giảm hoặc biến động của các thông số cảm biến.

b) Mô tả chi tiết

Chức năng cho phép người dùng theo dõi sự thay đổi của dữ liệu cảm biến theo thời gian thông qua biểu đồ trên Dashboard. Người dùng không cần lựa chọn thời gian bắt đầu và kết thúc mà chỉ cần lựa chọn loại cảm biến và số lượng lần đo gần nhất muốn xem.

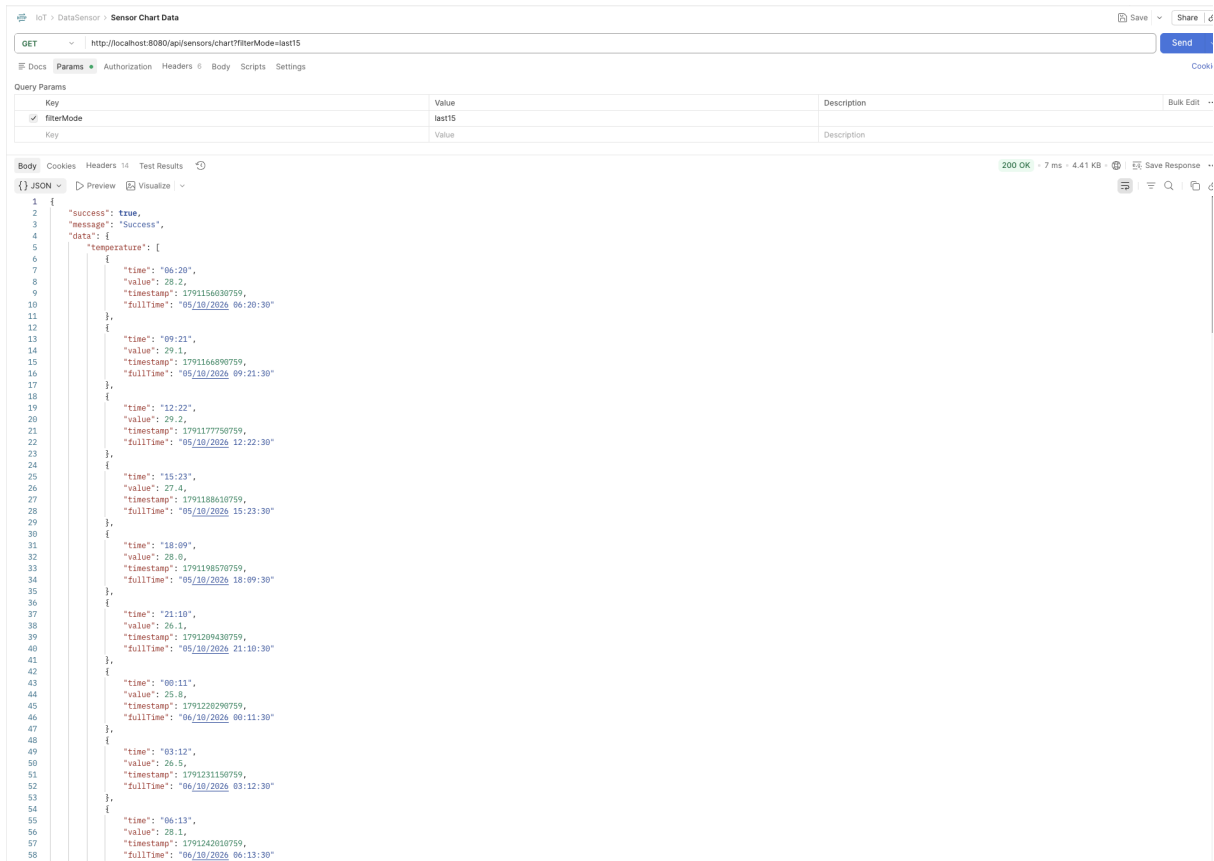
Luồng xử lý:

1. Người dùng truy cập vào **Dashboard**.
2. Người dùng lựa chọn loại cảm biến cần xem, ví dụ **Temperature**.
3. Người dùng lựa chọn số lượng lần đo gần nhất, ví dụ **15 lần đo gần nhất**.
4. Người dùng nhấn nút **Statistic**.
5. Frontend gửi request đến Backend thông qua RESTful API với loại cảm biến và số lượng bản ghi cần lấy.
6. Backend tiếp nhận request và kiểm tra thông tin xác thực JWT.
7. Backend yêu cầu SensorDataRepository truy vấn dữ liệu từ Database.
8. Database tìm các bản ghi thuộc loại cảm biến được yêu cầu và lấy **N lần đo mới nhất** dựa trên thời gian đo.
9. Database trả kết quả về Backend.
10. Backend xử lý và trả danh sách dữ liệu cảm biến về Frontend.
11. Frontend nhận dữ liệu và biểu diễn dưới dạng **biểu đồ đường (Line Chart)**.
12. Người dùng theo dõi sự thay đổi của giá trị cảm biến theo thời gian.

Các thông tin được hiển thị gồm:

- Loại cảm biến.
- Giá trị cảm biến.
- Thời gian đo.
- Số lượng lần đo được lựa chọn.
- Biểu đồ đường thể hiện sự thay đổi của giá trị cảm biến theo thời gian.

API đề xuất: GET api/sensors/chart



Hình 3.15 API Xem Biểu Đồ

Thuộc tính	Mô tả
Endpoint	/api/sensors/chart

Method	GET
Mục đích	Lấy N dữ liệu cảm biến gần nhất theo loại cảm biến được lựa chọn.
Authentication	Có – yêu cầu JWT Token.
Request Body	Không có
Query Parameters	filter mode

Bảng 3.12 Thông tin API xem biểu đồ

Request Header:

Authorization: Bearer <JWT_TOKEN>

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả	Ví dụ
type	String	Có	Loại cảm biến cần xem	temperature
limit	Integer	Có	Số lượng lần đo gần nhất cần lấy	15

Bảng 3.13 Tham số Request của API xem biểu đồ

Các trường dữ liệu trả về

Trường	Kiểu dữ liệu	Mô tả
type	String	Loại cảm biến
value	Double	Giá trị đo được
timestamp	DateTime	Thời gian đo

Bảng 3.14 Các trường dữ liệu trả về API xem biểu đồ

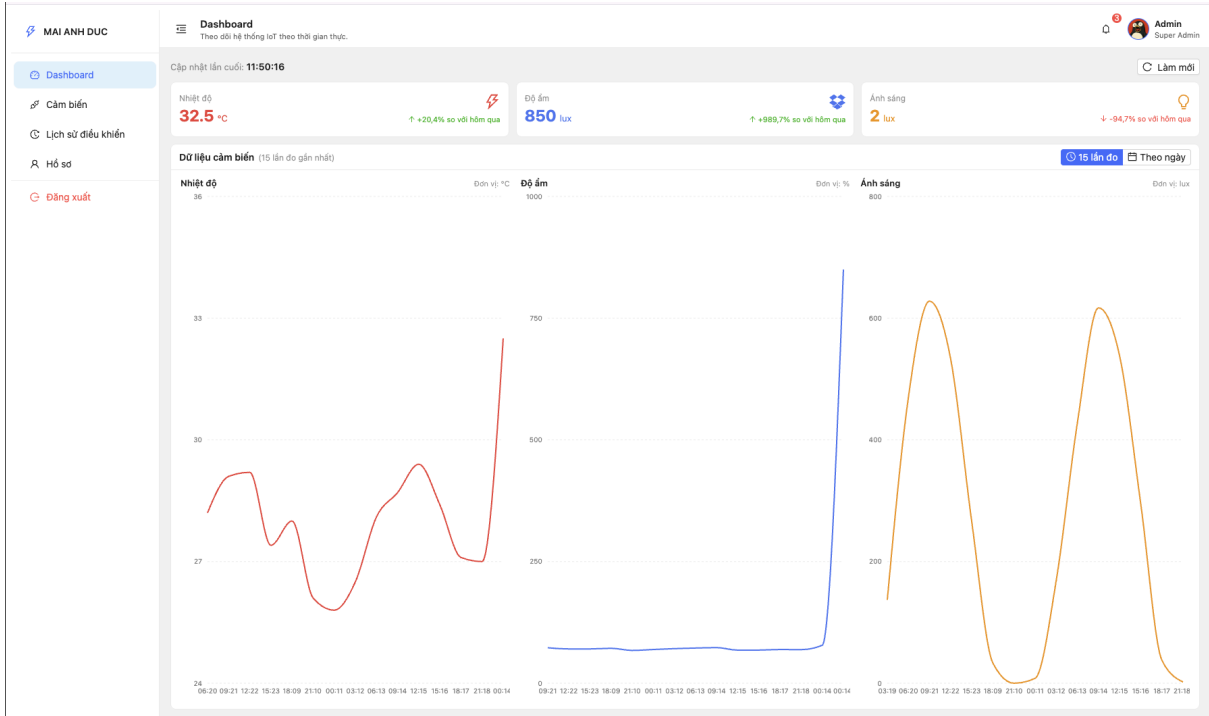
Các mã trạng thái trả về

Mã	Tên	Mô tả
200	OK	Truy vấn dữ liệu thành công.
400	Bad Request	Tham số truyền vào không hợp lệ.
401	Unauthorized	Chưa xác thực hoặc JWT Token không hợp lệ/hết hạn.
403	Forbidden	Không có quyền truy cập API.

500	Internal Server Error	Lỗi xảy ra phía Backend/Server.
-----	-----------------------	---------------------------------

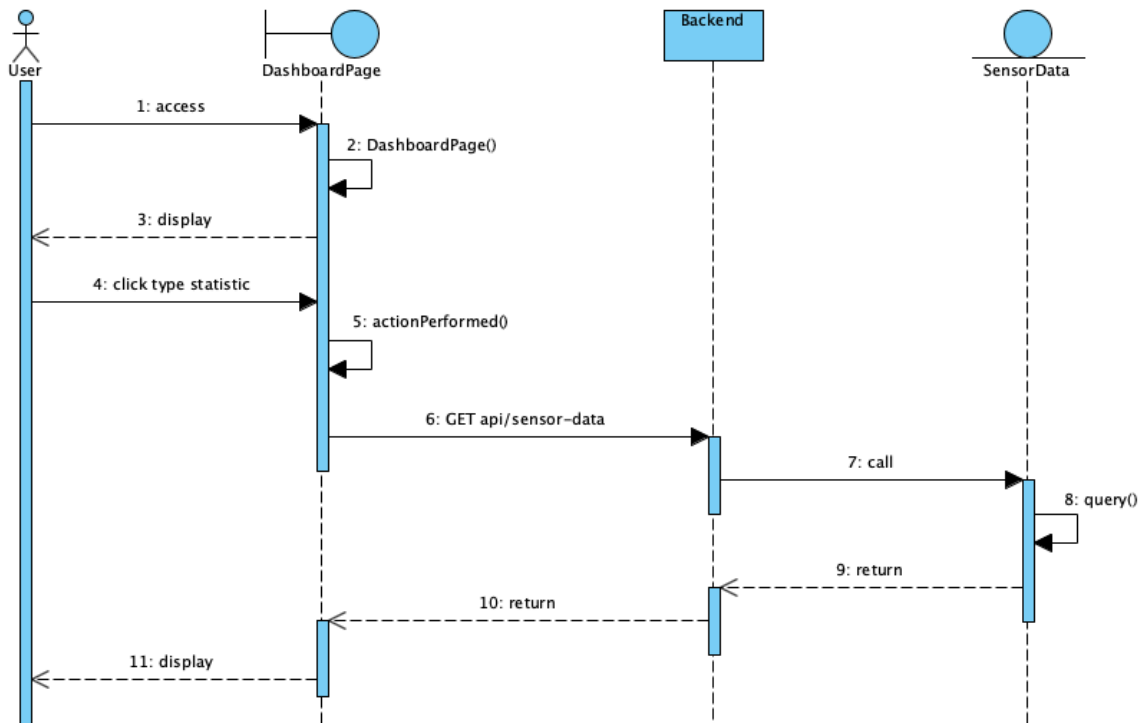
Bảng 3.15 Các mã trả về của API xem biểu đồ

c) Giao diện người dùng



Hình 3.16 Giao diện xem biểu đồ dữ liệu cảm biến

d) Sequence diagram



Hình 3.17 Sequence Diagram Xem biểu đồ

Mô tả tuần tự

1. User truy cập Dashboard
User gửi yêu cầu access đến DashboardPage.
2. DashboardPage khởi tạo giao diện
DashboardPage thực hiện DashboardPage() để tải và khởi tạo màn hình Dashboard.
3. Hiển thị Dashboard
Sau khi khởi tạo xong, DashboardPage hiển thị giao diện cho User.
4. User chọn loại thống kê
User thực hiện thao tác click type statistic trên Dashboard, ví dụ chọn xem dữ liệu nhiệt độ, độ ẩm hoặc ánh sáng.
5. DashboardPage xử lý thao tác
DashboardPage thực hiện actionPerformed() để xử lý lựa chọn của User.

6. Frontend yêu cầu dữ liệu từ Backend

DashboardPage gửi request: GET /api/sensor-data đến Backend để lấy dữ liệu cảm biến tương ứng.

7. Backend yêu cầu lấy dữ liệu

Backend gọi SensorData để yêu cầu truy vấn dữ liệu cảm biến.

8. SensorData thực hiện truy vấn

SensorData thực hiện query() để lấy dữ liệu cần thiết.

9. SensorData trả kết quả cho Backend

Sau khi truy vấn xong, dữ liệu cảm biến được trả về Backend.

10. Backend trả dữ liệu cho DashboardPage

Backend xử lý kết quả và trả dữ liệu cảm biến về cho Frontend.

11. Hiển thị dữ liệu cho User

DashboardPage nhận dữ liệu và hiển thị kết quả thống kê cho User.

3. Xem lịch sử cảm biến

a) Mô tả tính năng

Chức năng Xem lịch sử cảm biến cho phép người dùng xem lại các dữ liệu cảm biến đã được hệ thống thu thập và lưu trữ trong Database.

Người dùng có thể tìm kiếm dữ liệu theo loại cảm biến và khoảng thời gian, từ đó theo dõi lại các giá trị đã đo trong quá khứ.

b) Mô tả chi tiết

Luồng xử lý:

1. Người dùng truy cập vào giao diện Data Sensor.
2. Người dùng lựa chọn loại cảm biến cần xem.
3. Người dùng lựa chọn ngày và thời gian bắt đầu.
4. Người dùng lựa chọn ngày và thời gian kết thúc.
5. Người dùng nhấn nút Tìm kiếm.
6. Frontend gửi request đến Backend thông qua RESTful API với các điều kiện tìm kiếm.

- Backend tiếp nhận request và truy vấn Database.
- Database tìm kiếm các bản ghi cảm biến phù hợp với loại cảm biến và khoảng thời gian được yêu cầu.
- Database trả kết quả về cho Backend.
- Backend trả danh sách dữ liệu về Frontend.
- Frontend hiển thị lịch sử dữ liệu cảm biến dưới dạng bảng.

Các thông tin được hiển thị gồm:

- ID dữ liệu.
- Tên/loại cảm biến.
- Giá trị đo được.
- Đơn vị đo.
- Thời gian đo.

API đề xuất: GET api/sensors/data

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** `http://localhost:8080/api/sensors/data?sensorType=All&quickSearch=&page=1&size=3&sortBy=time&sortDir=desc`
- Query Params:**

Key	Value	Description
sensorType	All	
quickSearch		
page	1	
size	3	
sortBy	time	
sortDir	desc	
- Response (JSON):**

```

{
  "success": true,
  "message": "Success",
  "data": {
    "content": [
      {
        "id": "123",
        "sensor": "Light Sensor",
        "type": "Humidity",
        "value": 850.0,
        "unit": "lux",
        "status": "Error",
        "connection": "Online",
        "time": "07/18/2026 00:14:09",
        "timestamp": 1791306849074
      },
      {
        "id": "122",
        "sensor": "Humidity Sensor",
        "type": "Humidity",
        "value": 78.0,
        "unit": "H",
        "status": "Warning",
        "connection": "Online",
        "time": "07/18/2026 00:14:09",
        "timestamp": 1791306849074
      },
      {
        "id": "121",
        "sensor": "Temperature Sensor",
        "type": "Temperature",
        "value": 32.5,
        "unit": "C",
        "status": "Warning",
        "connection": "Online",
        "time": "07/18/2026 00:14:09",
        "timestamp": 1791306849074
      }
    ],
    "page": 1,
    "size": 3,
    "totalElements": 123,
    "totalPages": 41,
    "last": false
  }
}

```

Hình 3.18 API Lấy dữ liệu cảm biến

Thuộc tính	Mô tả
Method	GET
Endpoint	/api/sensors/data
Mô tả	Lấy danh sách dữ liệu cảm biến đã được lưu trong Database theo loại cảm biến và khoảng thời gian.
Authentication	Có – yêu cầu JWT Token
Request Body	Không có
Query Parameters	sensorType, quickSearch, page, size, sortBy, sortDir

Bảng 3.16 Thông tin API xem lịch sử dữ liệu cảm biến

Query Parameters

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
sensorType	String	Không	Loại cảm biến (All, temperature, humidity, light)

quickSearch	String	Không	Tìm kiếm nhanh
page	Integer	Không	Số trang cần lấy
size	Integer	Không	Số bản ghi trên một trang
sortBy	String	Không	Trường cần sắp xếp
sortDir	String	Không	asc hoặc desc

Bảng 3.17 Tham số Request API xem lịch sử dữ liệu cảm biến

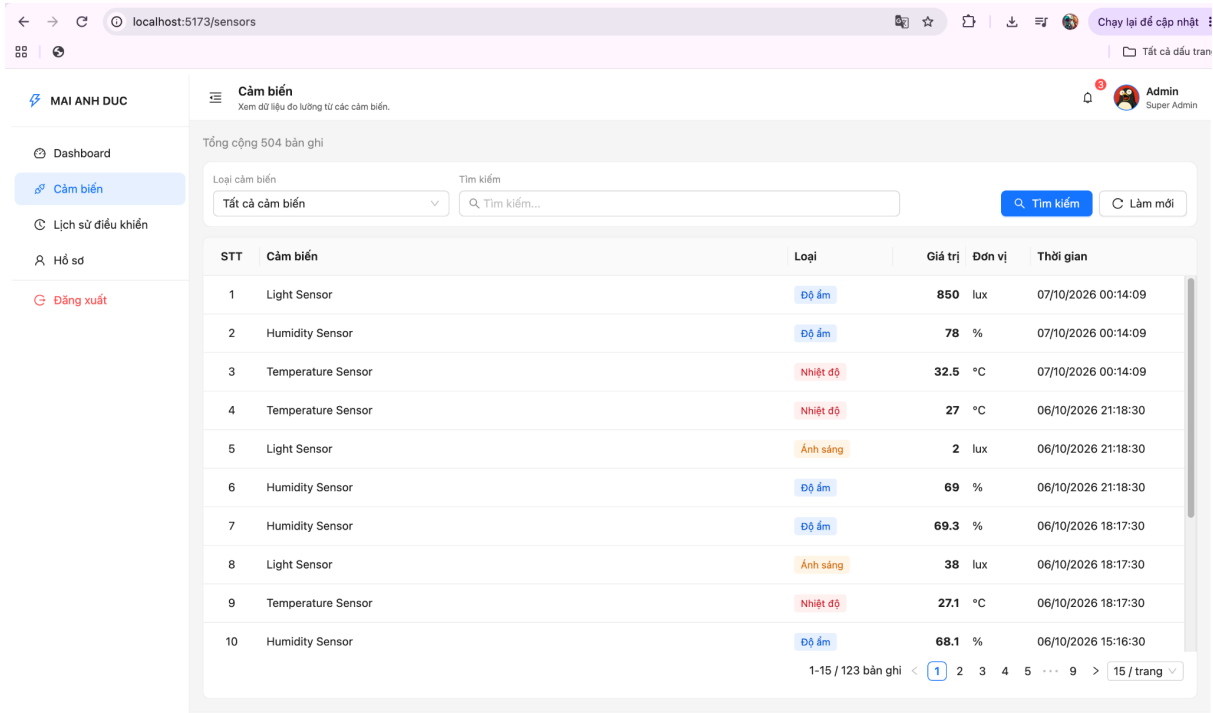
Các mã trả về

Mã	Ý nghĩa
200 OK	Lấy dữ liệu thành công
400 Bad Request	Tham số truyền vào không hợp lệ
401 Unauthorized	Chưa xác thực hoặc JWT không hợp lệ
403 Forbidden	Không có quyền truy cập

404 Not Found	Không tìm thấy dữ liệu
500 Internal Server Error	Lỗi phía Server

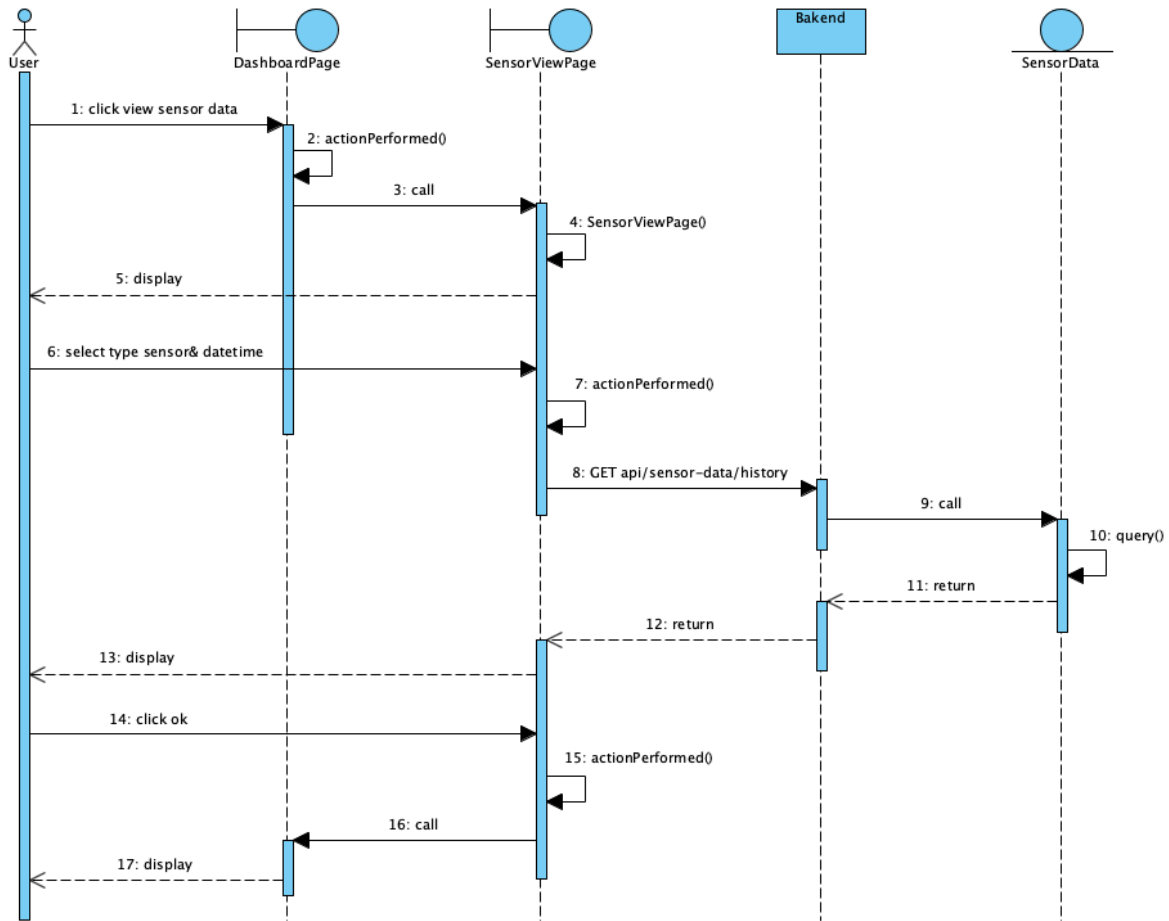
Bảng 3.18 Các mã trả về của API xem dữ liệu cảm biến

c) Giao diện người dùng



Hình 3.19 Giao diện xem lịch sử cảm biến

d) Sequence diagram



Hình 3.20 Sequence Diagram Xem dữ liệu cảm biến

Mô tả tuần tự

1. Người dùng nhấn “View Sensor Data” trên DashboardView để truy cập chức năng xem dữ liệu cảm biến.
2. DashboardView tiếp nhận thao tác của người dùng và thực hiện xử lý sự kiện.
3. DashboardView gửi yêu cầu mở SensorViewPage.
4. SensorViewPage được khởi tạo để cung cấp giao diện xem dữ liệu cảm biến.
5. SensorViewPage hiển thị giao diện xem dữ liệu cảm biến cho người dùng.
6. Người dùng chọn loại cảm biến, ngày bắt đầu, ngày kết thúc, thời gian bắt đầu, thời gian kết thúc và nhấn OK.
7. SensorViewPage tiếp nhận thao tác và xử lý yêu cầu tìm kiếm.
8. SensorViewPage gửi request GET /api/sensor-data/history đến Backend với các điều kiện tìm kiếm.

9. Backend tiếp nhận request và gọi DataSensor để lấy dữ liệu phù hợp.
10. DataSensor truy vấn dữ liệu cảm biến theo loại cảm biến và khoảng thời gian được yêu cầu
11. DataSensor trả kết quả dữ liệu cảm biến về Backend.
12. Backend trả danh sách dữ liệu cảm biến về ViewHistorySensor.
13. ViewHistorySensor hiển thị lịch sử dữ liệu cảm biến cho người dùng.
14. Người dùng nhấn OK sau khi hoàn tất việc xem lịch sử cảm biến.
15. ViewHistorySensor tiếp nhận thao tác và xử lý sự kiện.
16. ViewHistorySensor gửi yêu cầu quay lại SensorViewPage.
17. SensorViewPage hiển thị lại giao diện cho người dùng.

III. Điều khiển thiết bị

1. Bật tắt thiết bị

a. Mô tả chức năng

Chức năng **Bật/Tắt thiết bị** cho phép người dùng điều khiển trạng thái của các thiết bị IoT từ giao diện Dashboard. Người dùng có thể yêu cầu bật hoặc tắt thiết bị thông qua nút **ON/OFF** tương ứng.

Trước khi thực hiện thao tác điều khiển, hệ thống sẽ gọi API để **kiểm tra trạng thái hiện tại của thiết bị**. Dựa trên trạng thái nhận được, hệ thống xác định thiết bị đã ở trạng thái mà người dùng yêu cầu hay chưa.

- Nếu thiết bị **đã ở trạng thái được yêu cầu**, hệ thống không gửi lại lệnh điều khiển và thông báo cho người dùng.
- Nếu thiết bị **chưa ở trạng thái được yêu cầu**, hệ thống gửi lệnh điều khiển đến Backend. Backend publish lệnh thông qua **MQTT Broker** đến ESP32.
- ESP32 thực hiện bật/tắt thiết bị và gửi **trạng thái thực tế** về Backend.
- Backend xác nhận kết quả, cập nhật trạng thái thiết bị và lưu lịch sử điều khiển.

Chức năng đảm bảo trạng thái hiển thị trên giao diện phản ánh trạng thái thực tế của thiết bị.

b. Mô tả chi tiết

Luồng xử lý:

1. Người dùng truy cập Dashboard và lựa chọn thiết bị cần điều khiển.
2. Người dùng nhấn nút ON/OFF tương ứng với thiết bị.
3. DashboardPage xác định deviceId và trạng thái mà người dùng muốn thực hiện.
4. Frontend gọi API kiểm tra trạng thái hiện tại của thiết bị: GET
`/api/devices/{id}/status`
5. Backend tiếp nhận request và xác thực JWT Token.

Ngoại lệ: Nếu JWT Token không hợp lệ hoặc hết hạn, Backend trả về 401 Unauthorized.

6. Backend kiểm tra thiết bị có tồn tại trong thực thể Device hay không.

Ngoại lệ: Nếu không tìm thấy thiết bị, Backend trả về 404 Not Found.

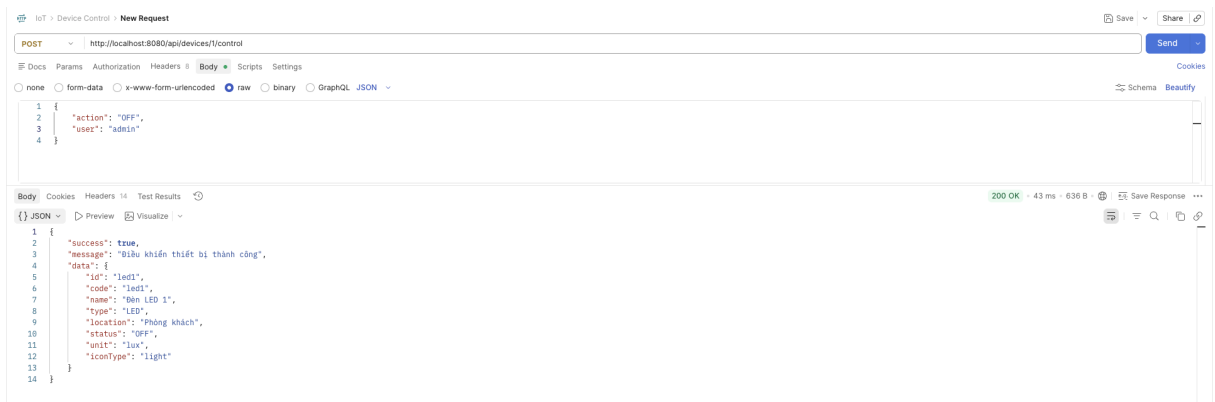
7. Backend lấy trạng thái hiện tại của thiết bị và trả về Frontend.
8. Frontend nhận trạng thái hiện tại và so sánh với trạng thái mà người dùng yêu cầu.
9. Người dùng click ON/OFF dựa vào trạng thái của thiết bị
10. Backend tiếp nhận yêu cầu điều khiển.
11. Backend kiểm tra lại thông tin thiết bị và quyền điều khiển.
12. Backend publish lệnh điều khiển lên MQTT Broker thông qua topic tương ứng với thiết bị.
13. MQTT Broker chuyển tiếp lệnh đến ESP32 đang subscribe topic điều khiển.
14. ESP32 nhận lệnh và xử lý.
15. ESP32 điều khiển thiết bị vật lý thông qua GPIO.
16. Thiết bị thực hiện thao tác bật hoặc tắt.
17. ESP32 xác định trạng thái thực tế của thiết bị sau khi thực hiện.

18. ESP32 publish trạng thái thực tế lên MQTT Broker thông qua topic trạng thái:
19. Backend nhận trạng thái thực tế từ MQTT Broker.
20. Backend so sánh: Requested Status = ON và Actual Status = ON. Nếu hai trạng thái giống nhau → điều khiển thành công.
21. Nếu điều khiển thành công, Backend cập nhật trạng thái mới vào thực thể Device.
22. Backend tạo bản ghi trong thực thể Action để lưu lịch sử điều khiển.
23. Backend trả kết quả điều khiển về Frontend.
24. Frontend cập nhật trạng thái thiết bị trên Dashboard.
25. Frontend hiển thị thông báo cho người dùng.

Các thông tin được hiển thị gồm:

- **Tên thiết bị:** Tên của thiết bị được người dùng lựa chọn để điều khiển.
- **Trạng thái hiện tại:** Trạng thái thực tế của thiết bị trước khi thực hiện lệnh, gồm ON hoặc OFF.
- **Trạng thái yêu cầu:** Trạng thái mà người dùng muốn thiết lập cho thiết bị, gồm ON hoặc OFF.
- **Hành động:** Hành động điều khiển tương ứng, gồm TURN_ON hoặc TURN_OFF.
- **Thời gian thực hiện:** Thời điểm thực hiện thao tác điều khiển, theo định dạng dd/MM/yyyy HH:mm:ss.
- **Kết quả điều khiển:** Kết quả thực hiện lệnh, gồm **Thành công**, **Thất bại** hoặc **Thiết bị đã ở trạng thái yêu cầu**.
- **Trạng thái sau điều khiển:** Trạng thái thực tế của thiết bị sau khi ESP32 thực hiện lệnh, gồm ON hoặc OFF.
- **Thông báo:** Thông báo kết quả thực hiện lệnh cho người dùng.

API đề xuất:



Hình 3.21 API điều khiển thiết bị

1. API 1: Kiểm tra trạng thái thiết bị: **GET /api/devices/{id}/status**

Thuộc tính	Mô tả
Endpoint	/api/devices/{id}/status
Method	GET
Mục đích	Kiểm tra trạng thái hiện tại của thiết bị (Ví dụ: ESP32, Relay, Đèn, Quạt...).
Authentication	Có – yêu cầu JWT Token.
Path Parameter	id (String/Int) – ID của thiết bị cần kiểm tra.

Request Body	Không có.
Query Parameters	Không có.

Bảng 3.19 Thông tin API lấy thông tin trạng thái của thiết bị

Các mã trạng thái trả về (Response Status Codes)

Mã	Tên (Status Name)	Mô tả (Description)
200	OK	Lấy trạng thái thiết bị thành công.
401	Unauthorized	Người dùng chưa đăng nhập, JWT Token không hợp lệ hoặc đã hết hạn.
403	Forbidden	Người dùng đã xác thực nhưng không có quyền truy cập thông tin thiết bị này.
404	Not Found	Không tìm thấy thiết bị với ID tương ứng trong hệ thống.
500	Internal Server Error	Có lỗi xảy ra phía Backend/Server trong quá trình xử lý.

Bảng 3.20 Các mã trả về API lấy trạng thái thiết bị

2. API 2: Điều khiển thiết bị: POST api/devices/{id}/control

Thuộc tính	Mô tả
Method	POST
Endpoint	/api/devices/{id}/control
Mô tả	Gửi lệnh bật hoặc tắt thiết bị thông qua Backend và MQTT Broker.
Authentication	Có – yêu cầu JWT Token
Path Parameters	id – ID của thiết bị cần điều khiển
Request Body	Có
Content-Type	application/json

Bảng 3.21 Thông tin API điều khiển thiết bị

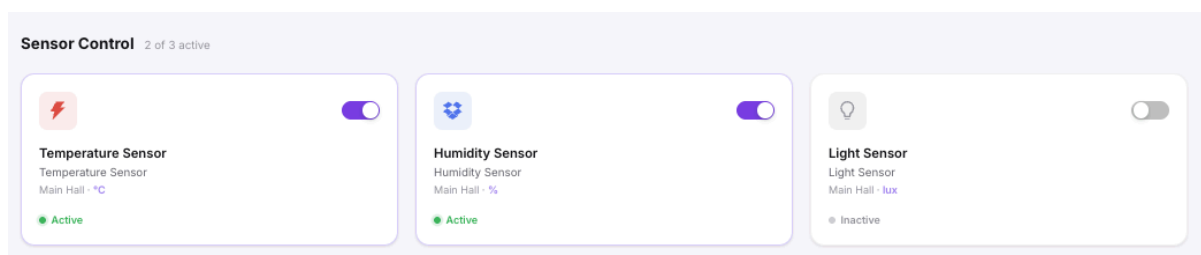
Các mã trả về

Mã	Ý nghĩa
----	---------

200 OK	Gửi và thực hiện lệnh điều khiển thành công
400 Bad Request	Action không hợp lệ hoặc dữ liệu request không hợp lệ
401 Unauthorized	Chưa xác thực hoặc JWT không hợp lệ
403 Forbidden	Người dùng không có quyền điều khiển thiết bị
404 Not Found	Không tìm thấy thiết bị
409 Conflict	Trạng thái thiết bị không phù hợp để thực hiện lệnh
500 Internal Server Error	Lỗi phía Backend/MQTT

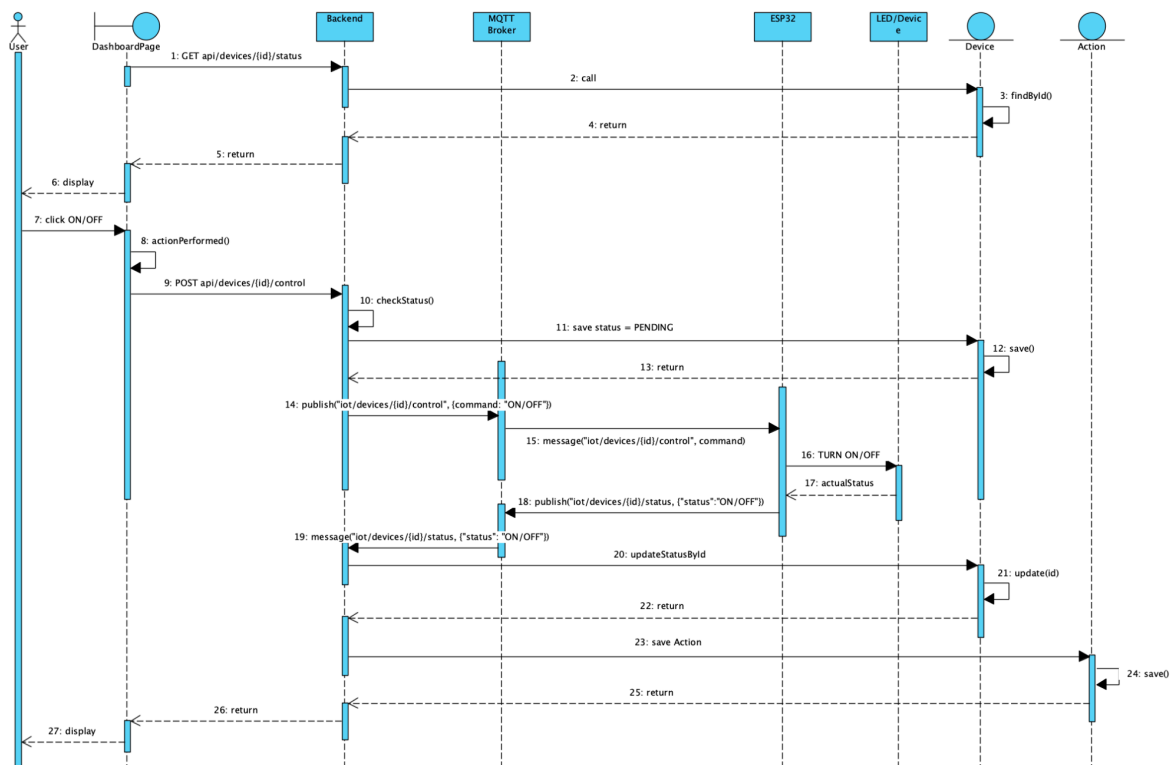
Bảng 3.22 Các mã trả về của API điều khiển thiết bị

c) Giao diện người dùng



Hình 3.22 Giao diện điều khiển thiết bị

d) Sequence diagram



Hình 3.23 Sequence Diagram điều khiển thiết bị

Mô tả tuần tự:

1. DashboardPage gửi request GET /api/devices/{id}/status đến Backend để lấy trạng thái hiện tại của thiết bị.
2. Backend gọi Device để lấy thông tin thiết bị theo id.
3. Device thực hiện findById() để truy vấn thông tin và trạng thái thiết bị trong cơ sở dữ liệu.
4. Device trả thông tin và trạng thái thiết bị về Backend.
5. Backend trả dữ liệu trạng thái về DashboardPage.
6. DashboardPage hiển thị trạng thái ON/OFF của thiết bị cho User.
7. User nhấn nút ON/OFF trên DashboardPage để điều khiển thiết bị.
8. DashboardPage thực hiện actionPerformed() để xử lý thao tác của User.
9. DashboardPage gửi request POST /api/devices/{id}/control đến Backend, kèm lệnh ON hoặc OFF.

10. Backend thực hiện checkStatus() để kiểm tra trạng thái thiết bị trước khi gửi lệnh điều khiển.
11. Backend yêu cầu Device lưu trạng thái xử lý yêu cầu điều khiển là PENDING.
12. Device thực hiện save() để lưu trạng thái PENDING vào cơ sở dữ liệu.
13. Device trả kết quả lưu về Backend.
14. Backend publish lệnh điều khiển lên MQTT Broker thông qua topic `iot/devices/{id}/control`, với nội dung `{"command": "ON"}` hoặc `{"command": "OFF"}`.
15. MQTT Broker chuyển message điều khiển đến ESP32 đã subscribe topic `iot/devices/{id}/control`.
16. ESP32 nhận lệnh và điều khiển LED/Device bật hoặc tắt tương ứng.
17. ESP32 xác định trạng thái sau điều khiển của LED/Device, được biểu diễn bằng phản hồi actualStatus trong sơ đồ.
18. ESP32 publish trạng thái thiết bị lên MQTT Broker thông qua topic `iot/devices/{id}/status`, với nội dung `{"status": "ON"}` hoặc `{"status": "OFF"}`.
19. MQTT Broker chuyển message trạng thái đến Backend đã subscribe topic `iot/devices/{id}/status`.
20. Backend gọi updateStatusById() trên Device để cập nhật trạng thái thiết bị theo id dựa trên phản hồi của ESP32.
21. Device thực hiện update(id) để lưu trạng thái mới vào cơ sở dữ liệu.
22. Device trả kết quả cập nhật về Backend.
23. Backend gọi Action để lưu lịch sử điều khiển, gồm thiết bị, lệnh điều khiển, trạng thái kết quả và thời gian thực hiện.
24. Action thực hiện save() để lưu lịch sử điều khiển vào cơ sở dữ liệu.
25. Action trả kết quả lưu về Backend.
26. Backend trả kết quả điều khiển và trạng thái mới về DashboardPage.
27. DashboardPage hiển thị trạng thái mới và thông báo bật/tắt thiết bị thành công cho User.

2. Xem lịch sử điều khiển thiết bị

a) Mô tả tính năng

Cho phép người dùng xem lại lịch sử các thao tác bật/tắt thiết bị trong hệ thống. Thông tin lịch sử bao gồm **tên thiết bị, hành động, trạng thái và thời gian thực hiện**, giúp người dùng theo dõi và kiểm tra các hoạt động điều khiển thiết bị đã diễn ra.

b) Mô tả chi tiết

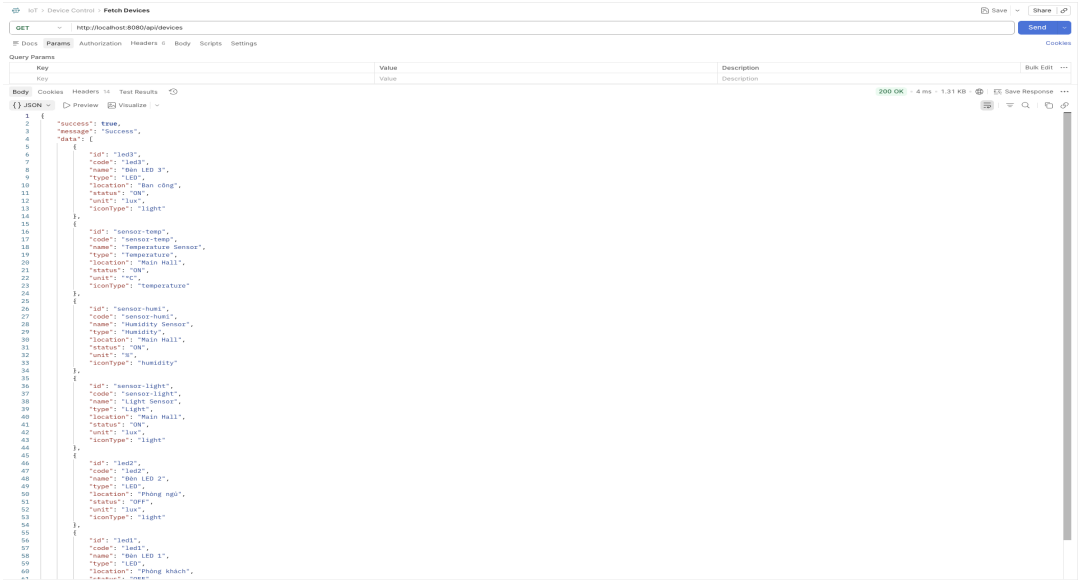
Luồng xử lý:

1. Người dùng chọn chức năng Xem lịch sử điều khiển trên giao diện Lịch sử điều khiển.
2. Frontend gửi yêu cầu lấy lịch sử điều khiển đến Backend thông qua RESTful API.
3. Backend tiếp nhận và xử lý yêu cầu.
4. Backend truy vấn Database để lấy danh sách lịch sử điều khiển của các thiết bị.
5. Database thực hiện truy vấn bảng ControlHistory và trả về các bản ghi lịch sử.
6. Backend tiếp nhận và xử lý dữ liệu lịch sử từ Database.
7. Backend trả danh sách lịch sử điều khiển về Frontend.
8. Frontend nhận dữ liệu và hiển thị lịch sử điều khiển trên giao diện cho người dùng.
9. Người dùng có thể xem, theo dõi các lần bật/tắt thiết bị đã được thực hiện.

Các thông tin được hiển thị gồm:

- Tên thiết bị: Tên của thiết bị đã được điều khiển.
- Trạng thái: Trạng thái của thiết bị sau khi thực hiện lệnh, ON hoặc OFF.
- Hành động: Hành động đã thực hiện, TURN_ON hoặc TURN_OFF.
- Thời gian thực hiện: Ngày, giờ, phút, giây theo định dạng dd/MM/yyyy HH:mm:ss.
- Kết quả điều khiển: Kết quả của thao tác, SUCCESS hoặc FAILED.
- Thông báo: Thông tin mô tả kết quả hoặc trạng thái thực hiện thao tác.

API đề xuất: GET api/devices



Hình 3.24 API xem lịch sử điều khiển thiết bị

Thuộc tính	Mô tả
Method	GET
Endpoint	/api/devices
Mô tả	Lấy danh sách lịch sử các thao tác điều khiển thiết bị đã được lưu trong Database.
Authentication	Có – yêu cầu JWT Token
Request Body	Không có

Query Parameters	deviceId, from, to, page, size
------------------	--------------------------------

Bảng 3.23 Thông tin API xem lịch sử điều khiển thiết bị

Query Parameters

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
deviceId	Long	Không	ID thiết bị cần xem lịch sử
from	DateTime	Không	Thời gian bắt đầu
to	DateTime	Không	Thời gian kết thúc
page	Integer	Không	Số trang
size	Integer	Không	Số bản ghi trên một trang

Bảng 3.24 Tham số Request của API xem lịch sử điều khiển thiết bị

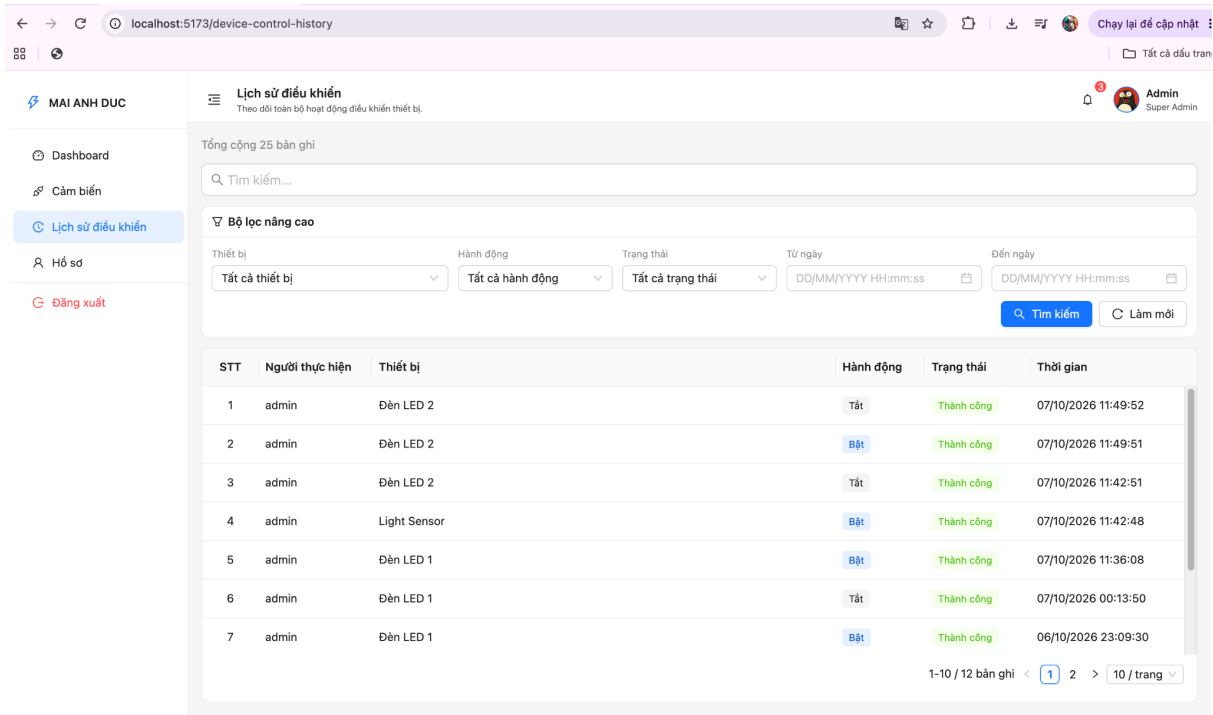
Các mã trả về

Mã	Ý nghĩa
200 OK	Lấy lịch sử điều khiển thành công

400 Bad Request	Tham số tìm kiếm không hợp lệ
401 Unauthorized	Chưa xác thực hoặc JWT không hợp lệ
403 Forbidden	Không có quyền xem lịch sử
404 Not Found	Không tìm thấy dữ liệu lịch sử
500 Internal Server Error	Lỗi phía Backend/Server

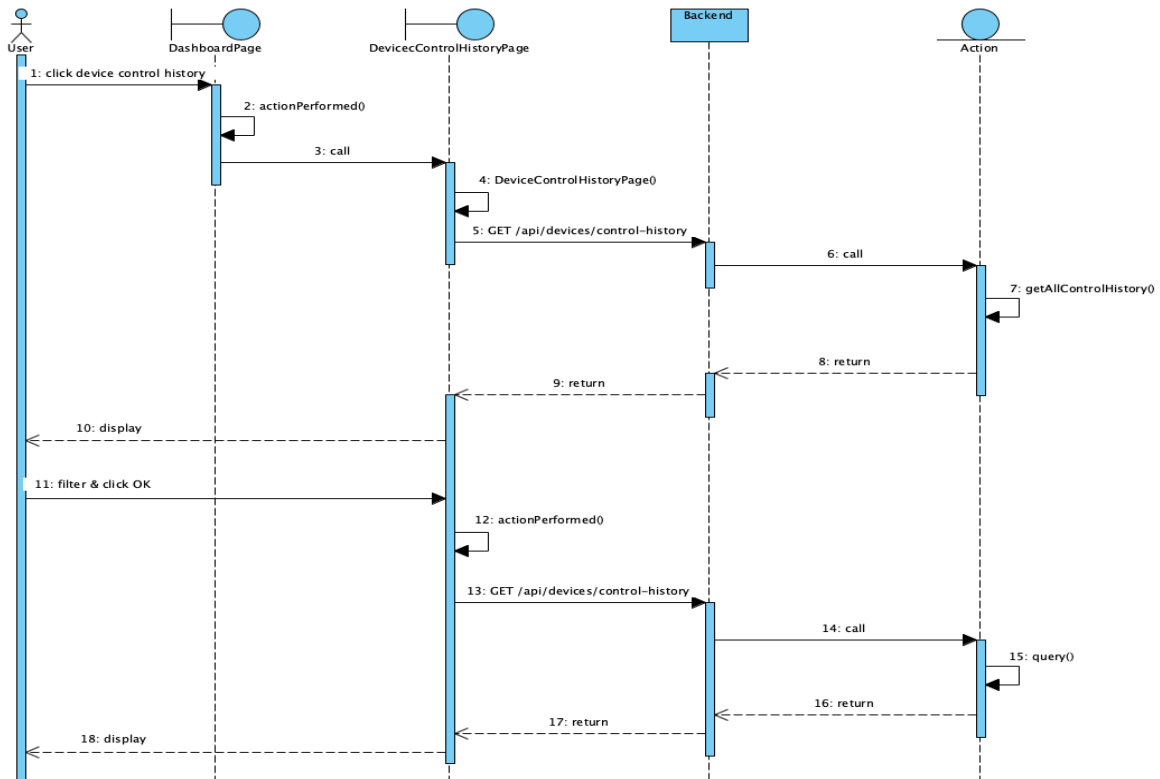
Bảng 3.25 Các mã trả về của API xem lịch sử điều khiển thiết bị

c) Giao diện người dùng



Hình 3.25 Giao diện xem lịch sử điều khiển thiết bị

d) Sequence diagram



Hình 3.26 Sequence Diagram xem lịch sử điều khiển thiết bị

Mô tả tuần tự:

1. Người dùng thao tác nhấn chọn mục xem lịch sử điều khiển thiết bị từ giao diện trang tổng quan.
2. Hệ thống ghi nhận sự kiện tương tác của người dùng.
3. Giao diện tổng quan thực hiện lệnh chuyển hướng sang trang chi tiết lịch sử điều khiển.
4. Giao diện trang lịch sử điều khiển thiết bị bắt đầu được khởi tạo.
5. Ngay trong quá trình khởi tạo, giao diện này tự động gửi một yêu cầu (HTTP GET) đến máy chủ (Backend) để lấy toàn bộ dữ liệu lịch sử.
6. Backend tiếp nhận yêu cầu và tiến hành gọi xuống Cơ sở dữ liệu.
7. Cơ sở dữ liệu thực thi truy vấn để quét và thu thập toàn bộ các bản ghi về lịch sử điều khiển.
8. Cơ sở dữ liệu trả danh sách dữ liệu vừa tìm được lại cho Backend.

9. Backend tiếp nhận, xử lý dữ liệu và gửi phản hồi kết quả về lại cho giao diện người dùng.
10. Giao diện hiển thị danh sách toàn bộ lịch sử điều khiển lên màn hình để người dùng theo dõi.
11. Người dùng thao tác trên bộ lọc của giao diện, chọn các tiêu chí mong muốn như: loại thiết bị, hành động (bật/tắt), mốc thời gian bắt đầu và kết thúc.
12. Giao diện ghi nhận sự thay đổi từ bộ lọc mà người dùng vừa thiết lập.
13. Dựa trên các tiêu chí đó, giao diện tự động tạo một yêu cầu mới, đính kèm các điều kiện lọc này vào đường dẫn API và gửi đến Backend.
14. Backend nhận yêu cầu và truyền chính xác các điều kiện lọc này xuống Cơ sở dữ liệu.
15. Cơ sở dữ liệu thực thi một truy vấn có điều kiện để quét và chỉ lấy ra những bản ghi khớp với yêu cầu.
16. Cơ sở dữ liệu trả danh sách dữ liệu đã được lọc lặp lại cho Backend.
17. Backend hoàn tất quá trình xử lý và trả gói dữ liệu lịch sử này về cho giao diện.
18. Giao diện cập nhật lại bảng dữ liệu trên màn hình, lúc này chỉ hiển thị những bản ghi thỏa mãn chính xác các tiêu chí tìm kiếm của người dùng.

CHƯƠNG 4. TỔNG KẾT VÀ LỜI CẢM ƠN

4.1. Tổng kết

Sau quá trình tìm hiểu, phân tích, thiết kế và xây dựng, em đã hoàn thành đề tài “Hệ thống IoT giám sát môi trường và điều khiển thiết bị từ xa” với các chức năng cơ bản đáp ứng được mục tiêu ban đầu đặt ra.

Hệ thống được xây dựng dựa trên sự kết hợp giữa phần cứng và phần mềm. Ở phía phần cứng, ESP32 được sử dụng làm bộ điều khiển trung tâm, kết nối với các cảm biến để thu thập các thông số môi trường như nhiệt độ, độ ẩm và cường độ ánh sáng. Bên cạnh đó, ESP32 còn thực hiện điều khiển các thiết bị đầu ra như đèn LED dựa trên các lệnh được gửi từ hệ thống.

Giao thức MQTT được sử dụng làm phương thức truyền thông giữa ESP32 và Backend. Thông qua mô hình Publish/Subscribe, dữ liệu cảm biến có thể được truyền từ thiết bị lên hệ thống, đồng thời các lệnh điều khiển cũng có thể được gửi từ Backend xuống ESP32. Nhờ đó, hệ thống có khả năng giao tiếp hai chiều giữa thiết bị IoT và phần mềm quản lý.

Backend của hệ thống được xây dựng bằng Spring Boot, có nhiệm vụ tiếp nhận và xử lý dữ liệu từ MQTT Broker, lưu trữ thông tin vào cơ sở dữ liệu PostgreSQL, cung cấp các RESTful API cho Frontend và xử lý các yêu cầu điều khiển thiết bị từ người dùng.

Frontend được xây dựng bằng React, cung cấp giao diện trực quan giúp người dùng có thể theo dõi dữ liệu cảm biến, xem biểu đồ, tra cứu lịch sử dữ liệu, theo dõi trạng thái thiết bị và thực hiện các thao tác bật hoặc tắt thiết bị từ xa.

Cơ sở dữ liệu PostgreSQL được sử dụng để lưu trữ thông tin người dùng, thiết bị, cảm biến, dữ liệu cảm biến và lịch sử điều khiển. Việc lưu trữ dữ liệu giúp hệ thống không chỉ hiển thị thông tin tại thời điểm hiện tại mà còn hỗ trợ tra cứu và theo dõi dữ liệu trong quá khứ.

Thông qua quá trình thực hiện đề tài, em đã có cơ hội vận dụng nhiều kiến thức liên quan đến Internet of Things, lập trình vi điều khiển, giao thức MQTT, phát triển Backend, Frontend, thiết kế cơ sở dữ liệu và xây dựng RESTful API. Đồng thời, em cũng hiểu rõ hơn về cách các thành phần trong một hệ thống IoT phối hợp với nhau, từ tầng phần cứng, tầng truyền thông đến tầng ứng dụng.

Kết quả đạt được của đề tài bao gồm các chức năng chính như:

- Thu thập dữ liệu nhiệt độ, độ ẩm và ánh sáng từ môi trường.
- Truyền dữ liệu từ ESP32 đến hệ thống thông qua MQTT.
- Lưu trữ dữ liệu cảm biến trong cơ sở dữ liệu PostgreSQL.
- Hiển thị dữ liệu cảm biến trên giao diện Web.
- Hiển thị dữ liệu dưới dạng biểu đồ theo thời gian.
- Theo dõi trạng thái thiết bị.

- Điều khiển bật hoặc tắt thiết bị từ xa.
- Nhận phản hồi trạng thái thực tế từ ESP32.
- Lưu và tra cứu lịch sử điều khiển thiết bị.

Bên cạnh những kết quả đã đạt được, hệ thống vẫn còn một số hạn chế do phạm vi đề tài và thời gian thực hiện có giới hạn. Số lượng cảm biến và thiết bị được sử dụng hiện tại còn ít, chủ yếu phục vụ cho mục đích nghiên cứu và mô phỏng. Hệ thống cũng chưa được triển khai với số lượng lớn thiết bị hoạt động đồng thời và chưa tích hợp nhiều chức năng tự động hóa nâng cao.

Trong tương lai, hệ thống có thể được tiếp tục phát triển bằng cách bổ sung thêm nhiều loại cảm biến và thiết bị điều khiển, tích hợp relay để điều khiển các thiết bị điện thực tế, xây dựng chức năng cảnh báo khi thông số môi trường vượt ngưỡng, lập lịch điều khiển thiết bị, gửi thông báo đến người dùng hoặc áp dụng các thuật toán Machine Learning và trí tuệ nhân tạo để phân tích dữ liệu và hỗ trợ tự động hóa.

Nhìn chung, đề tài đã giúp em hiểu rõ hơn quy trình xây dựng một hệ thống IoT hoàn chỉnh, từ việc kết nối phần cứng, truyền dữ liệu, xử lý ở Backend, lưu trữ trong cơ sở dữ liệu cho đến hiển thị và tương tác với người dùng thông qua Frontend. Đây là những kiến thức và kinh nghiệm thực tế có giá trị, tạo nền tảng để em tiếp tục nghiên cứu và phát triển các hệ thống IoT phức tạp hơn trong tương lai.

4.2. Lời cảm ơn

Trong quá trình học tập và thực hiện đề tài “Hệ thống IoT giám sát môi trường và điều khiển thiết bị từ xa”, em đã nhận được sự hướng dẫn, hỗ trợ và những góp ý quý báu để có thể hoàn thành đề tài.

Trước hết, em xin gửi lời cảm ơn chân thành và sâu sắc đến TS. Nguyễn Quốc Uy, giảng viên hướng dẫn môn học, đã tận tình giảng dạy, truyền đạt những kiến thức chuyên môn cần thiết và hướng dẫn em trong suốt quá trình học tập cũng như thực hiện đề tài.

Những kiến thức mà thầy truyền đạt về Internet of Things, thiết bị nhúng, giao thức truyền thông, cách tổ chức và xây dựng một hệ thống IoT đã giúp em có được nền tảng cần thiết để từng bước triển khai đề tài. Bên cạnh kiến thức chuyên môn, những nhận xét và góp ý của thầy cũng giúp em nhận ra những điểm còn hạn chế trong quá trình thực hiện, từ đó có thể điều chỉnh và hoàn thiện hệ thống tốt hơn.

Em xin chân thành cảm ơn TS. Nguyễn Quốc Uy đã dành thời gian hướng dẫn, giải đáp các thắc mắc và tạo điều kiện để em có thể vận dụng những kiến thức đã học vào một bài toán thực tế. Quá trình thực hiện đề tài đã giúp em có thêm nhiều kinh nghiệm về lập trình IoT, phát triển phần mềm cũng như cách phân tích và xây dựng một hệ thống có sự kết hợp giữa phần cứng và phần mềm.

Em cũng xin gửi lời cảm ơn đến Học viện Công nghệ Bưu chính Viễn thông cùng các thầy cô đã tạo ra môi trường học tập thuận lợi, cung cấp những kiến thức nền tảng và chuyên môn cần thiết để em có thể thực hiện và hoàn thành đề tài này.

Mặc dù em đã cố gắng trong quá trình nghiên cứu, xây dựng và hoàn thiện sản phẩm, tuy nhiên do kiến thức, kinh nghiệm và thời gian thực hiện còn hạn chế nên báo cáo và hệ thống chắc chắn vẫn còn những thiếu sót. Em rất mong nhận được những nhận xét và góp ý từ thầy để có thể tiếp tục hoàn thiện kiến thức cũng như cải thiện sản phẩm trong tương lai.

Một lần nữa, em xin gửi lời cảm ơn chân thành đến TS. Nguyễn Quốc Uy vì sự tận tình hướng dẫn và hỗ trợ trong suốt quá trình học tập và thực hiện đề tài.

Em xin chân thành cảm ơn!